# Estructuras de Datos

2026
Primer Cuatrimestre

# REPASO

# Ejemplo: Determinar el tiempo de hallar el fondo de una pila

Procedimiento:

```
public <E> E hallarFondo(Stack<E> pila) throws Exception {
        if (pila.isEmpty())
                throw new Exception("Pila Vacia");
        else {
                E resultado = pila.pop();
                while ( pila.isEmpty() == false ) {
                        resultado = pila.pop();
                return resultado;
        }
}
```

# Ejemplo: Determinar el tiempo de hallar el fondo de una pila

Entrada: pila

Tamaño del problema: n = cantidad de elementos de la pila

Tenemos que ver que determinar el tiempo de isEmpty() y de pop()

Viendo el código de la implementación (de cualquiera de las dos pilas que hicimos), vemos que tienen O(1)

Procedimiento:

```
public <E> E hallarFondo(Stack<E> pila) throws Exception {
        if (pila.isEmpty())
                throw new Exception("Pila Vacia");
        else {
                E resultado = pila.pop();
                while ( pila.isEmpty() == false )
                        resultado = pila.pop();
                return resultado;
        }
}
```

$c_1$

$c_2$

$c_3$

$c_4 \cdot (n-1)$

$c_5$

$c_6$

$T(n) = c_1 + \max(c_2, c_4 + (c_4 + c_5)(n-1) + c_4 + c_6) =$

$= c_1 + \max(k_1, k_2 n) \longrightarrow O(n)$

# Ejemplo

Hallar el máximo entre el primer elemento y el último elemento de una lista simplemente enlazada de números:

```
public float maximo_pri_ult(ListaSimplementeEnlazada<Float> lista ) {

        float resultado;

        float primero = lista.first().element();

        float ultimo = lista.last().element();

        if ( primero < ultimo ) resultado = primero;

        else resultado = ultimo;

        return resultado;

}
```

# Ejemplo

*Entrada: lista simplemente enlazada*

Hallar el máximo entre el primer elemento y el último elemento de una lista simplemente enlazada de números:

*Tamaño de la entrada: n = cantidad de elementos de lista*

```
public float maximo_pri_ult(ListaSimplementeEnlazada<Float> lista ) {

        float resultado;

        float primero = lista.first().element();

        float ultimo = lista.last().element();

        if ( primero < ultimo ) resultado = primero;

        else resultado = ultimo;

        return resultado;

}
```

$c_1$

$c_2$ - porque first tiene O(1) y element tiene O(1)

$c_3 n$ - porque last tiene O(n) y element tiene O(1)

$c_4$ - porque una condición es primitiva, $c_5$ - primitiva

$c_6$ -primitiva

$c_7$ -primitiva

¿Qué pasa si usamos una lista doblemente enlazada con punteros al primero y último?

$$T(n) = c_1 + c_2 + c_3 n + c_4 + \max(c_5, c_6) + c_7 =$$

$$= c_1 + c_2 + c_3 n + c_4 + c_8 + c_7 \longrightarrow \max(c_1, c_2, c_3 n, c_4, c_8, c_7) = O(n)$$

# Ejemplo

¿Y para una lista doblemente enlazada con centinelas al inicio y fin?

*Tamaño de la entrada: n = cantidad de elementos de lista*

```
public float maximo_pri_ult(ListaDoblementeEnlazada<Float> lista ) {
        float resultado;
        float primero = lista.first().element();
        float ultimo = lista.last().element();
        if ( primero < ultimo ) resultado = primero;
        else resultado = ultimo;
        return resultado;
}
```

$c_1$

$c_2$ - porque first tiene O(1) y element tiene O(1)

$c_3$ - porque last tiene O(1) y element tiene O(1)

$c_4$ - porque una condición es primitiva, $c_5$ - primitiva

$c_6$ -primitiva

$c_7$ -primitiva

$T(n) = c_1 + c_2 + c_3 + c_4 + max(c_5, c_6) + c_7 =$

$= c_1 + c_2 + c_3 + c_4 + c_8 + c_7 \longrightarrow max(c_1, c_2, c_3, c_4, c_8, c_7) = O(1)$

# Tiempo de ejecución de mapeo con hash abierto

**Entrada**: Tabla de hash de N buckets y n entradas.

- Tamaño de la entrada: n = cantidad de entradas del hash
- Suponemos que **no hay distribución uniforme de claves**
- Peor caso:
  - Todas las claves terminan en un único bucket
  - En este caso el bucket es una lista de tamaño n
  - $T_{get}(n) : O(1 + n)$
  - $T_{put}(n) : O(1 + n)$
  - $T_{remove}(n) : O(1 + n)$
  - Nota: El 1 en (1+n) proviene del tiempo de calcular el hash de la clave, el cual no depende de n

**Conclusión**: En una mala distribución en la tabla de las claves, el mapeo tiene orden lineal de la cantidad de entradas

# Tiempo de ejecución de mapeo con hash abierto

**Entrada**: Tabla de hash de N buckets y n entradas.

- Tamaño de la entrada: n = cantidad de entradas del hash
- Suponemos que **sí hay distribución uniforme de claves**
- Peor caso:
  - Cada bucket tiene n/N entradas
  - En este caso el bucket es una lista de tamaño n/N
  - $T_{get}(n) = O(1+n/N)$
  - $n/N =$  = factor de carga < 0,9
  - Luego, $T_{get}(n) = O(1+n/N) = O(1+\lambda) = O(1,9) = O(1)$

**Conclusión**: Bajo distribución uniforme de claves y factor de carga mantenido, las operaciones de inserción, búsqueda y eliminación del hash tienen orden 1

# Ejercicio

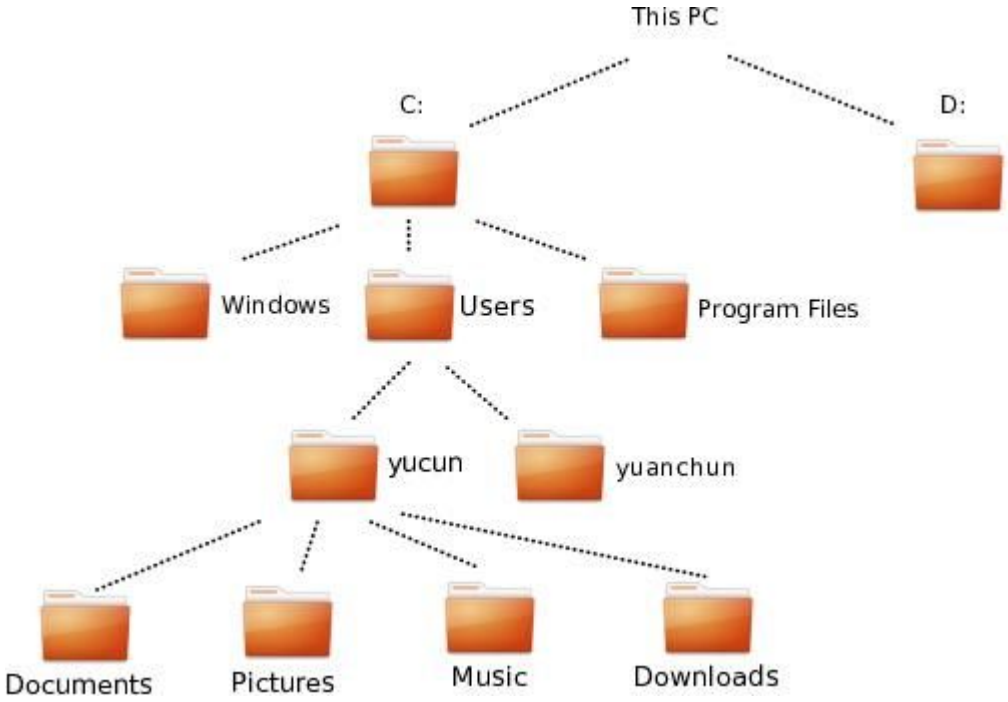
Calculen T(n) y a partir de ella su orden para cada implementación de los TDAs:

- Pila
- Cola
- Lista

En términos de tiempo de ejecución, ¿Cuál es la implementación más eficiente de cada uno de estos TDAs?

# TDA ÁRBOL

# Sistemas de archivos



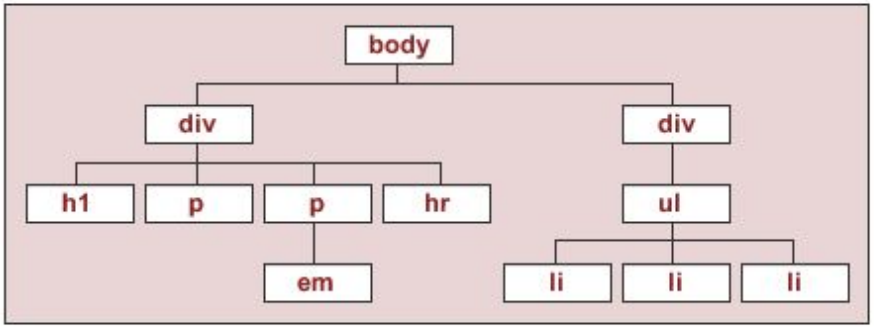
# Compiladores - HTML y DOM

```
<body>

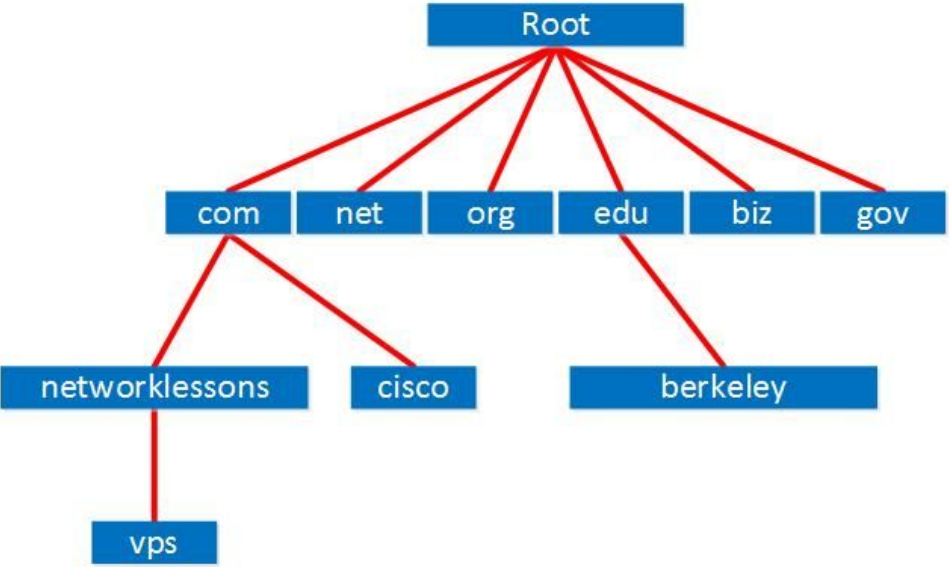
  <div id="content">
    <h1>Heading here</h1>
    <p>Lorem ipsum dolor sit amet.</p>
    <p>Lorem ipsum dolor <em>sit</em> amet.</p>
    <hr>
  </div>

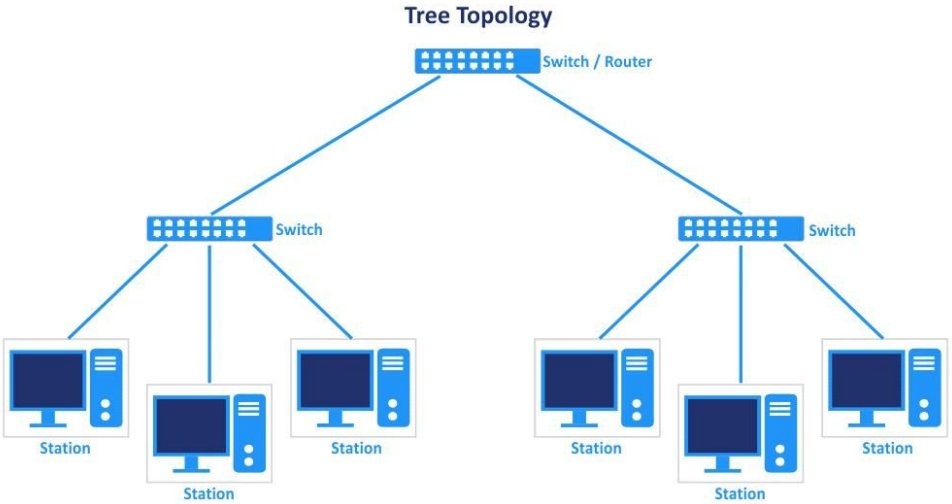
  <div id="nav">
    <ul>
      <li>item 1</li>
      <li>item 2</li>
      <li>item 3</li>
    </ul>
  </div>

</body>
```

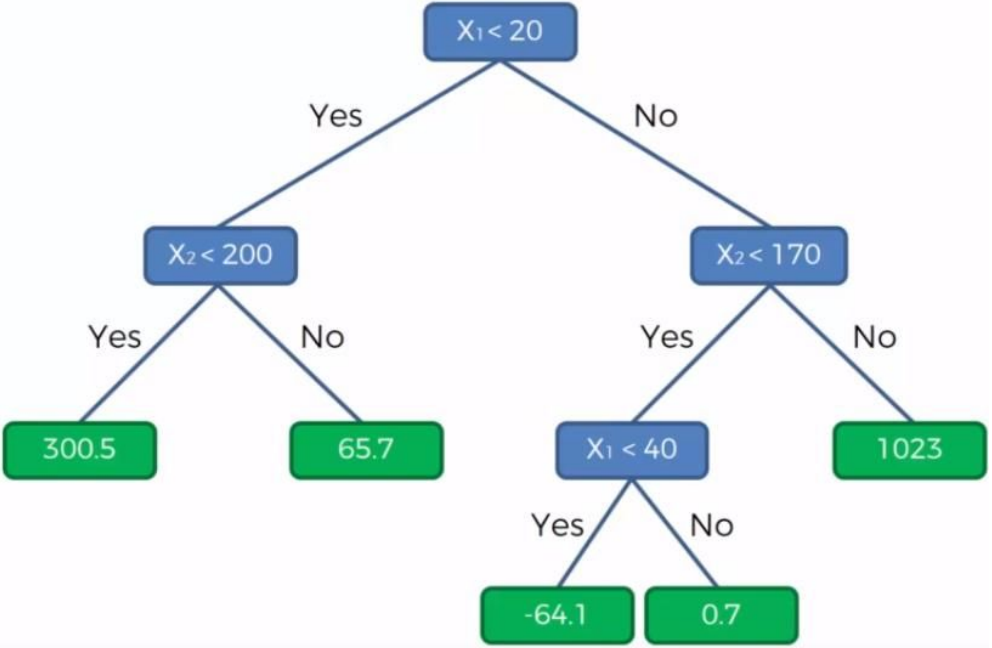


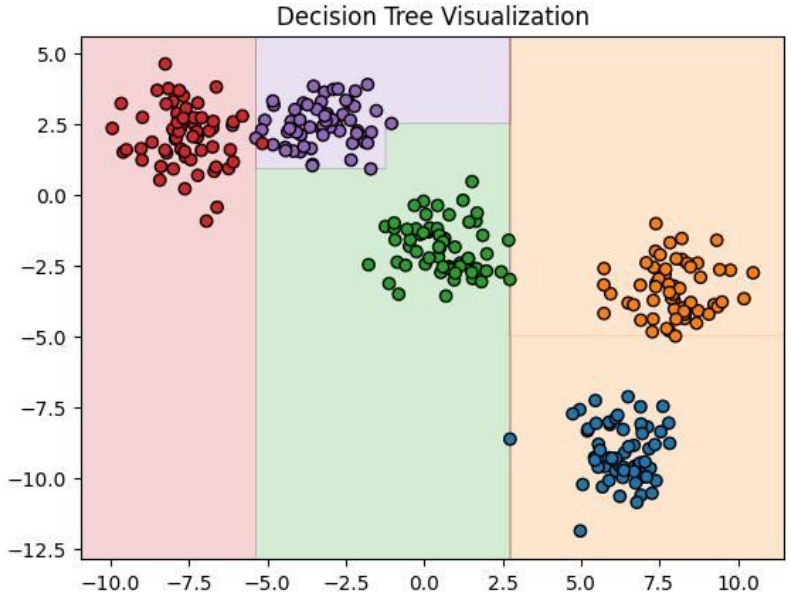
# Ruteo en redes



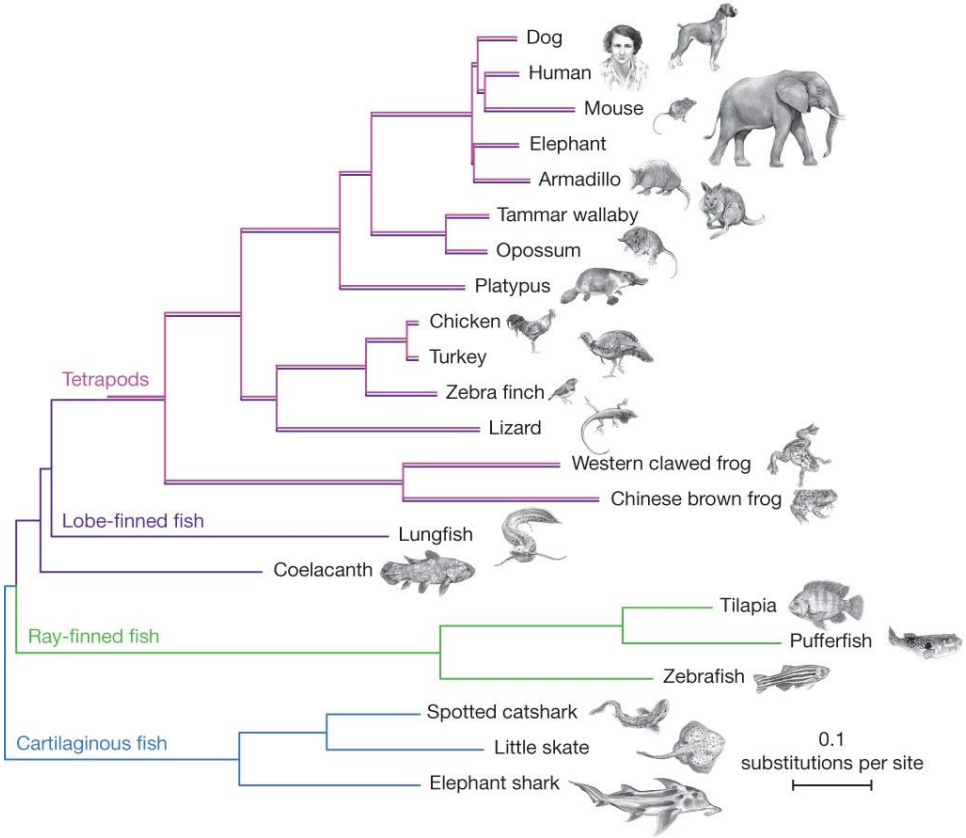



# Inteligencia artificial



# Biología - Filogenética



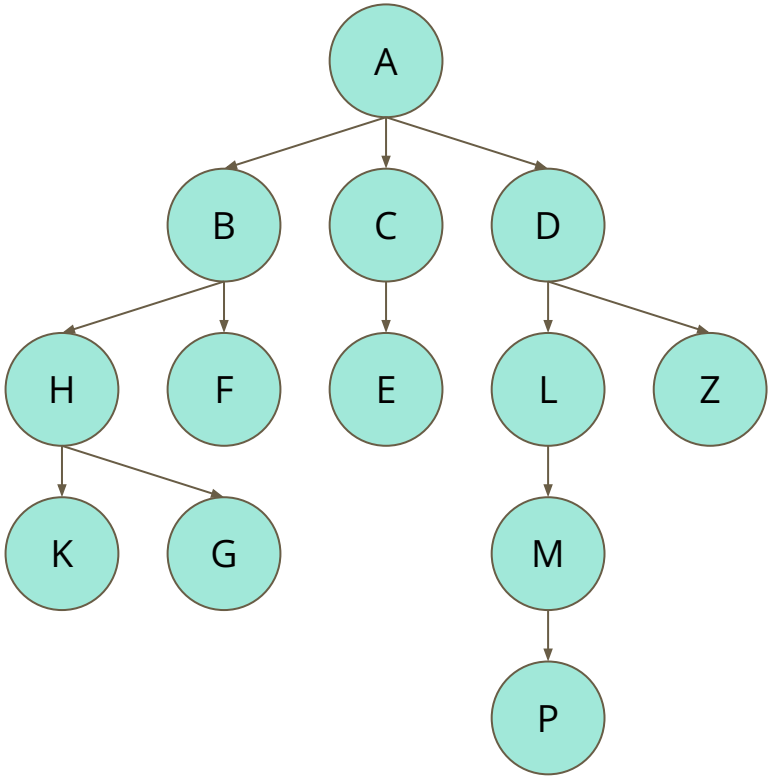

WORLD CUP 22
The Sporting News
NED
USA
ARG
AUS
NED
ARG
ARG
CRO
JAP
CRO
BRA
KOR
ARG
CHAMPION
ARGENTINA
THIRD PLACE
CROATIA
FOURTH PLACE
MOROCCO
FRA
ENG
SEN
FRA
POL
ENG
FRA
FRA
MAR
MAR
ESP
POR
SUI
POR
ROUND OF 16
QUARTERFINALS
SEMIFINALS
FINAL
SEMIFINALS
QUARTERFINALS
ROUND OF 16

# TDA ÁRBOL

Un árbol es un TDA que almacena los elementos llamados nodos jerárquicamente. Con la excepción del nodo raíz, cada nodo en un árbol tiene un nodo padre y cero o más nodos hijos.

# TDA ÁRBOL

La raíz se dibuja arriba.

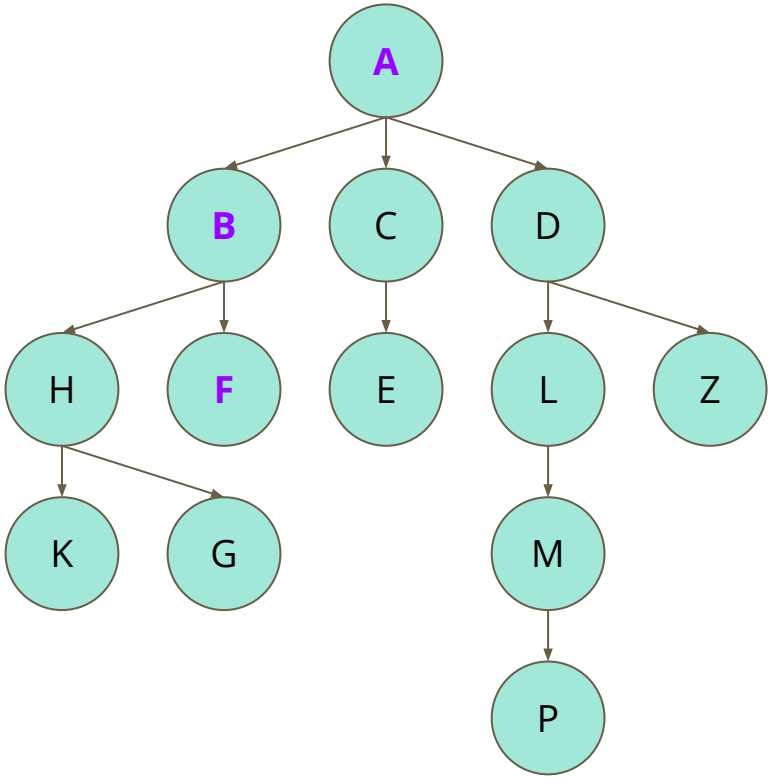


# TDA ÁRBOL

El nodo con rótulo A es la raíz.

El nodo con rótulo B es el padre de los nodos con rótulos H y F.

El nodo con rótulo F es el hijo del nodo con rótulo B



# TDA ÁRBOL

Un árbol T se define como un conjunto de nodos almacenando elementos tales que los nodos tienen una relación padre-hijo, que satisface:

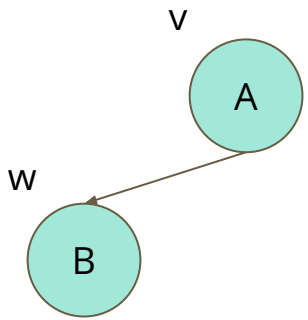
- Si T es no vacío, tiene un nodo especial, llamado la raíz de T, que no tiene padre.
- Cada nodo v de T diferente de la raíz tiene un único nodo padre w
- Cada nodo v con padre w es un hijo de w.

# TDA ÁRBOL

A es el rótulo de v.

B es el rótulo de w.

Cuando no haya confusión, nos referiremos al nodo con su rótulo

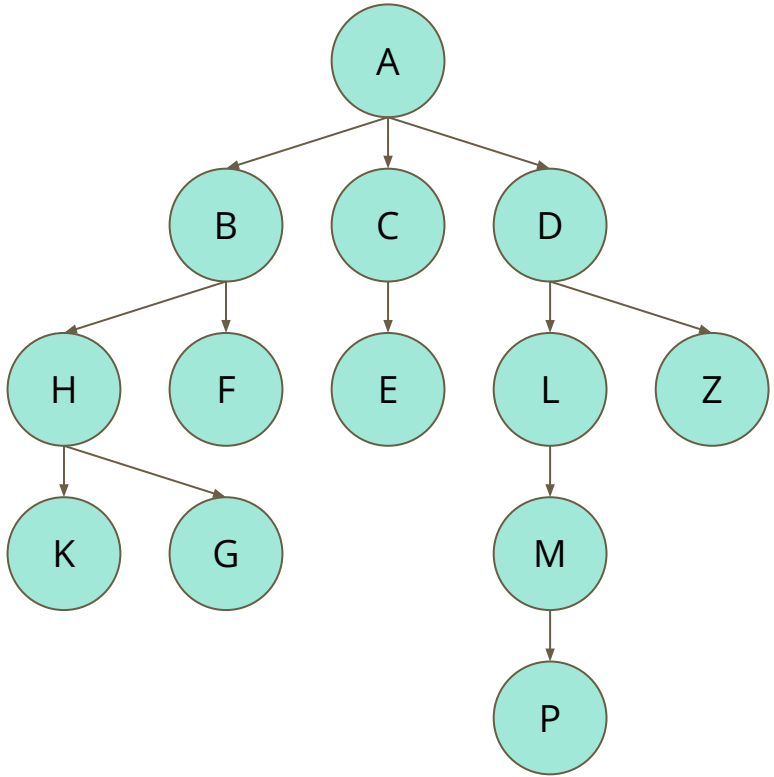


# TDA ÁRBOL - Definiciones

**Nodos Hermanos**: Dos nodos con el mismo padre se llaman hermanos. Ej. B y D son hermanos, L y Z lo son, H y E no son hermanos (son primos).

**Nodos externos u hojas**: Un nodo v es externo si no tiene hijos. Ej: F, K, G, E, P y Z son hojas.

**Nodo interno**: Un nodo v es interno si tiene uno o más hijos. Ej: A, B, C, D, H, L y M son nodos internos.

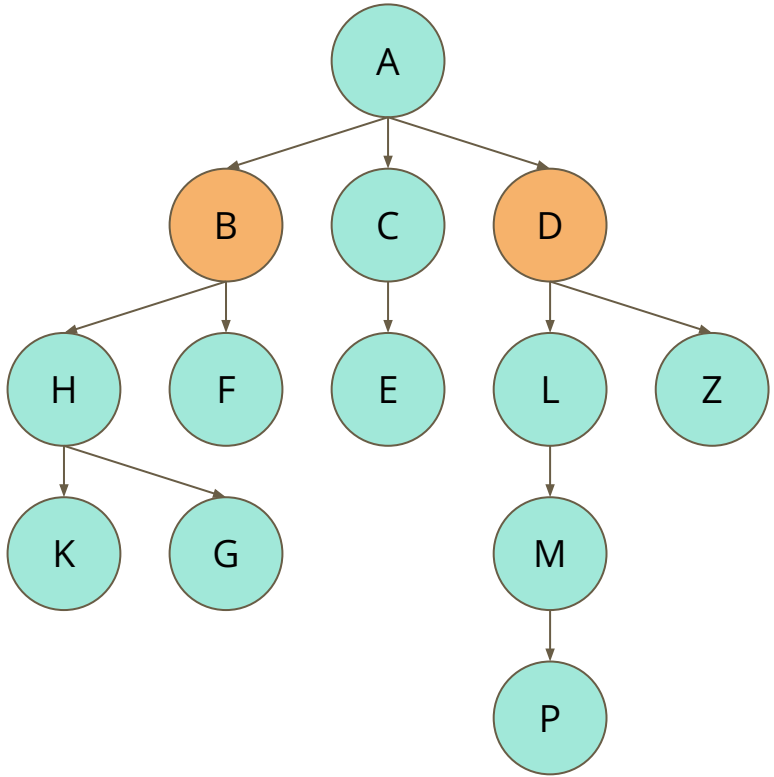


# TDA ÁRBOL - Definiciones

**Nodos Hermanos**: Dos nodos con el mismo padre se llaman hermanos. Ej. **B y D son hermanos**, L y Z lo son, H y E no son hermanos (son primos).

**Nodos externos u hojas**: Un nodo v es externo si no tiene hijos. Ej: F, K, G, E, P y Z son hojas.

**Nodo interno**: Un nodo v es interno si tiene uno o más hijos. Ej: A, B, C, D, H, L y M son nodos internos.

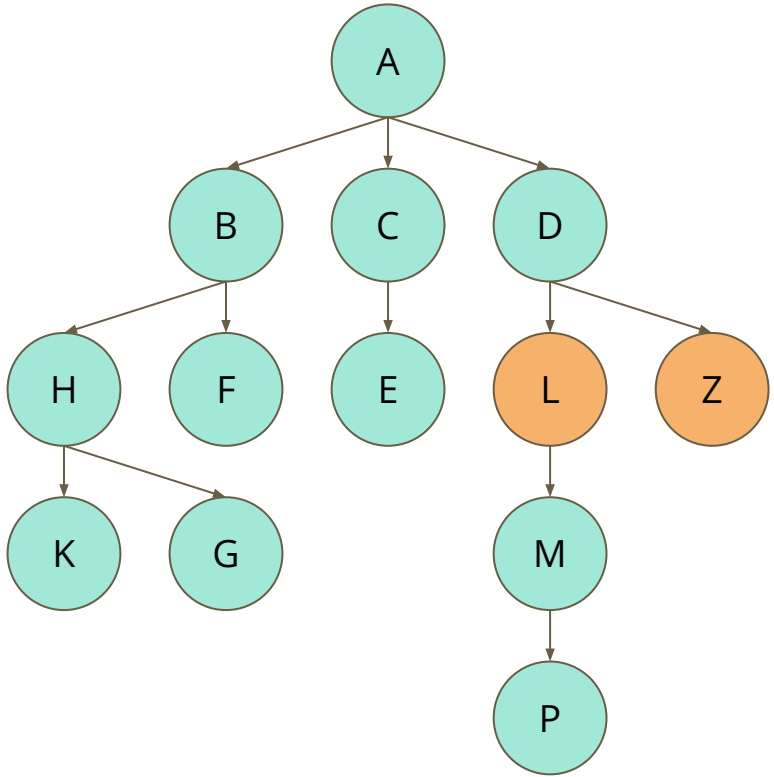


# TDA ÁRBOL - Definiciones

**Nodos Hermanos**: Dos nodos con el mismo padre se llaman hermanos. Ej. B y D son hermanos, **L y Z lo son**, H y E no son hermanos (son primos).

**Nodos externos u hojas**: Un nodo v es externo si no tiene hijos. Ej: F, K, G, E, P y Z son hojas.

**Nodo interno**: Un nodo v es interno si tiene uno o más hijos. Ej: A, B, C, D, H, L y M son nodos internos.

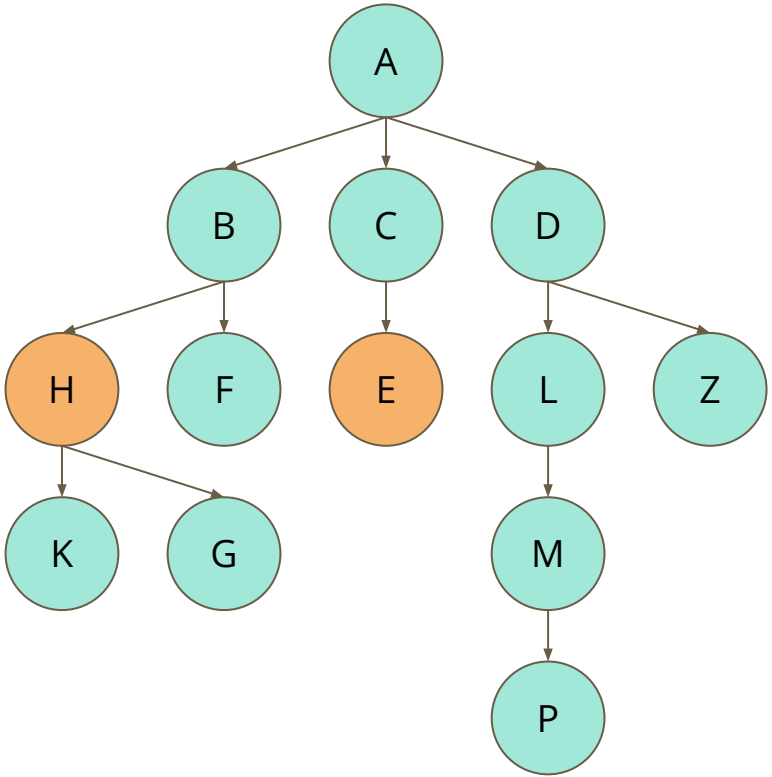


# TDA ÁRBOL - Definiciones

**Nodos Hermanos**: Dos nodos con el mismo padre se llaman hermanos. Ej. B y D son hermanos, L y Z lo son, **H y E no son hermanos (son primos).**

**Nodos externos u hojas**: Un nodo v es externo si no tiene hijos. Ej: F, K, G, E, P y Z son hojas.

**Nodo interno**: Un nodo v es interno si tiene uno o más hijos. Ej: A, B, C, D, H, L y M son nodos internos.

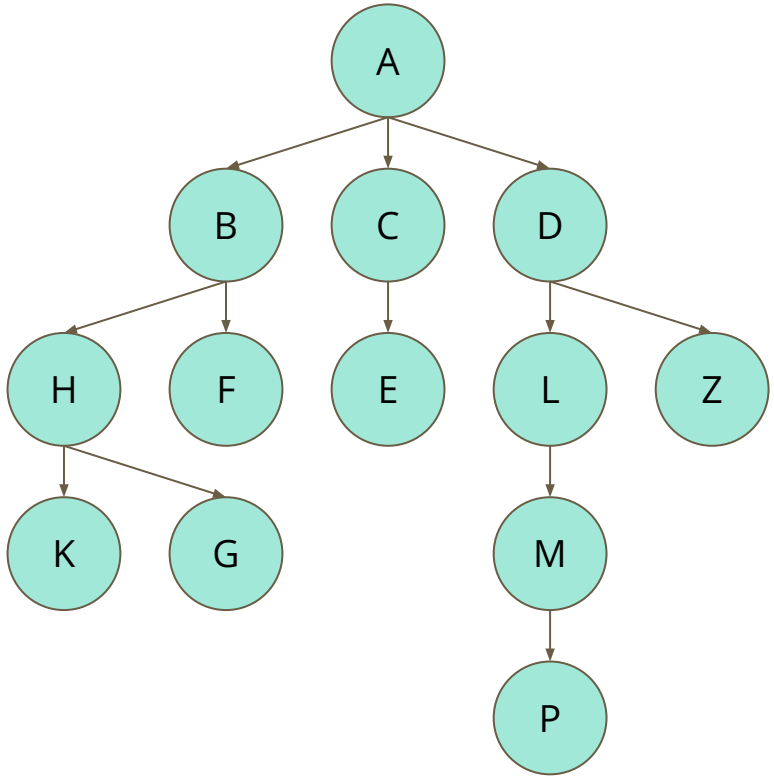


# TDA ÁRBOL - Definiciones

**Nodos Hermanos**: Dos nodos con el mismo padre se llaman hermanos. Ej. B y D son hermanos, L y Z lo son, H y E no son hermanos (son primos).

**Nodos externos u hojas**: Un nodo v es externo si no tiene hijos. Ej: F, K, G, E, P y Z son hojas.

**Nodo interno**: Un nodo v es interno si tiene uno o más hijos. Ej: A, B, C, D, H, L y M son nodos internos.

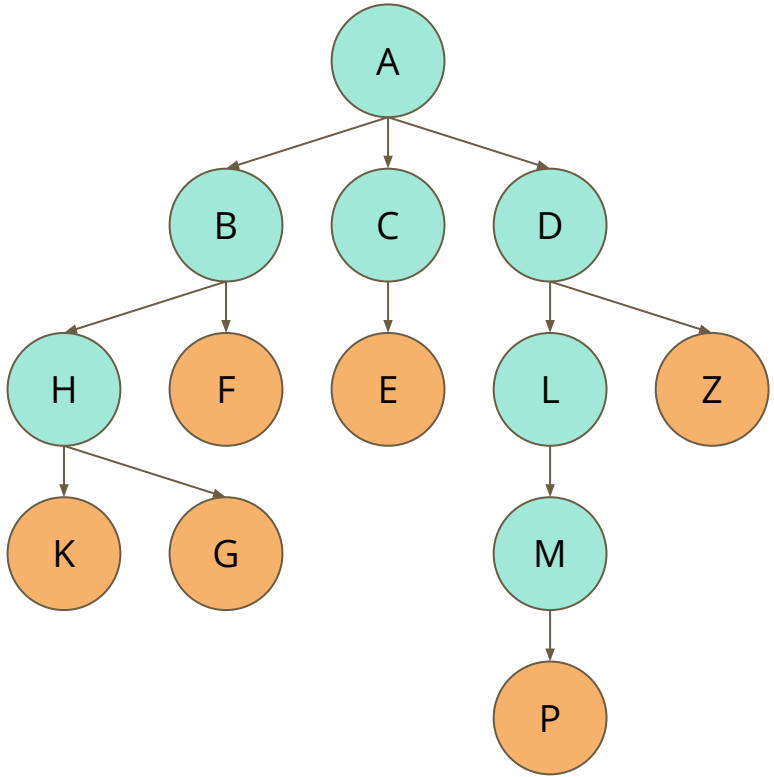


# TDA ÁRBOL - Definiciones

**Nodos Hermanos**: Dos nodos con el mismo padre se llaman hermanos. Ej. B y D son hermanos, L y Z lo son, H y E no son hermanos (son primos).

**Nodos externos u hojas**: Un nodo v es externo si no tiene hijos. Ej: **F, K, G, E, P y Z son hojas**.

**Nodo interno**: Un nodo v es interno si tiene uno o más hijos. Ej: A, B, C, D, H, L y M son nodos internos.

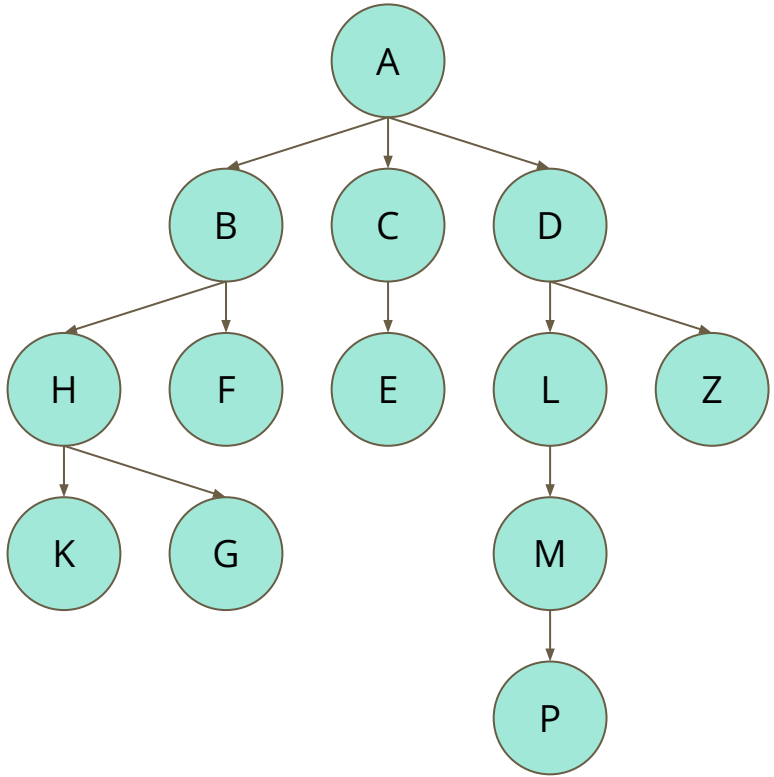


# TDA ÁRBOL - Definiciones

**Nodos Hermanos**: Dos nodos con el mismo padre se llaman hermanos. Ej. B y D son hermanos, L y Z lo son, H y E no son hermanos (son primos).

**Nodos externos u hojas**: Un nodo v es externo si no tiene hijos. Ej: F, K, G, E, P y Z son hojas.

**Nodo interno**: Un nodo v es interno si tiene uno o más hijos. Ej: A, B, C, D, H, L y M son nodos internos.

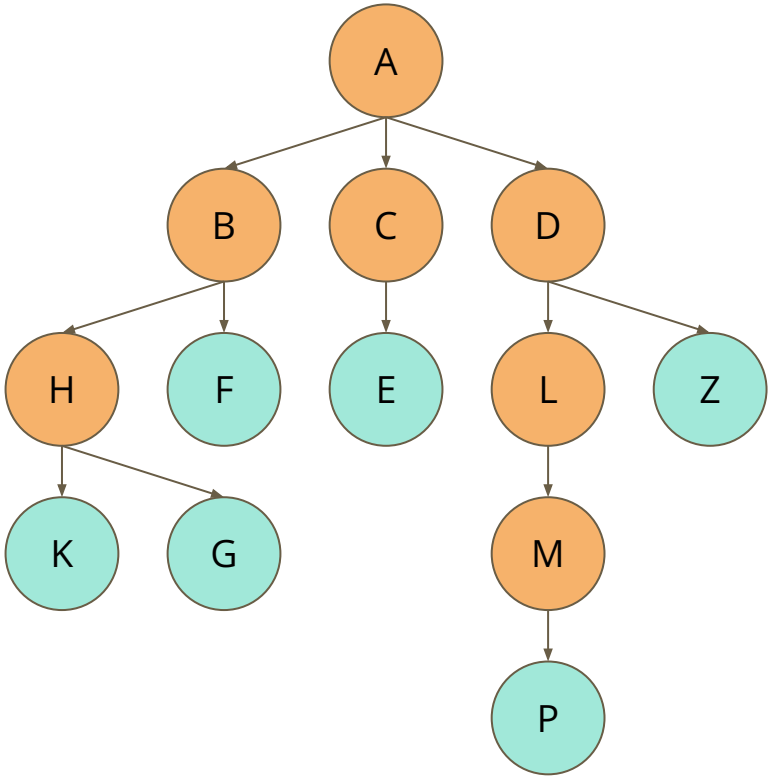


# TDA ÁRBOL - Definiciones

**Nodos Hermanos**: Dos nodos con el mismo padre se llaman hermanos. Ej. B y D son hermanos, L y Z lo son, H y E no son hermanos (son primos).

**Nodos externos u hojas**: Un nodo v es externo si no tiene hijos. Ej: F, K, G, E, P y Z son hojas.

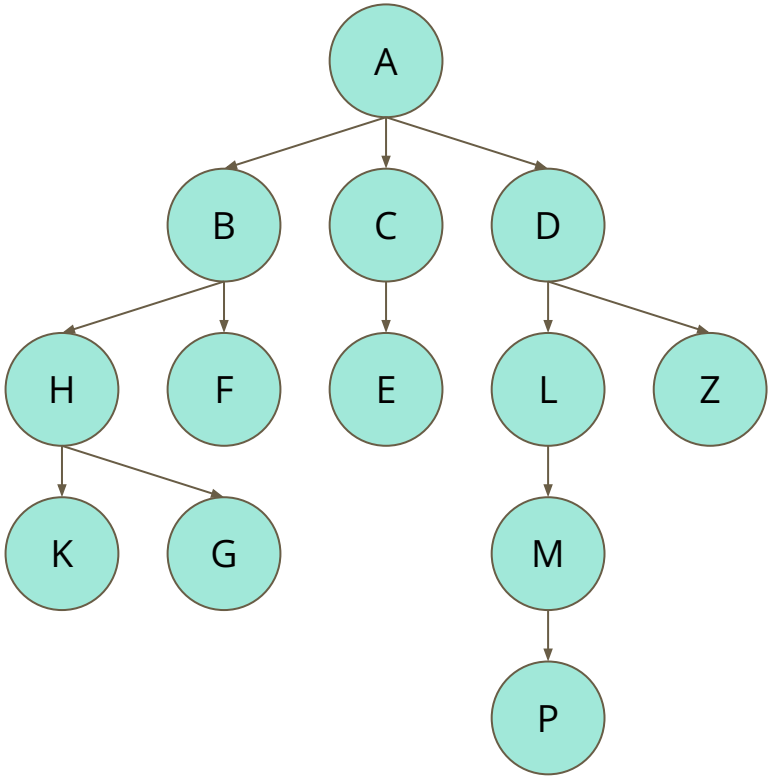
**Nodo interno**: Un nodo v es interno si tiene uno o más hijos. Ej: **A, B, C, D, H, L y M son nodos internos**.



# TDA ÁRBOL - Definiciones

**Ancestro(u,v)**: Un nodo u es ancestro de un nodo v si u=v o u es un ancestro del padre de v. Ej: ancestro(C,C), ancestro(A,E), ancestro(D,P)

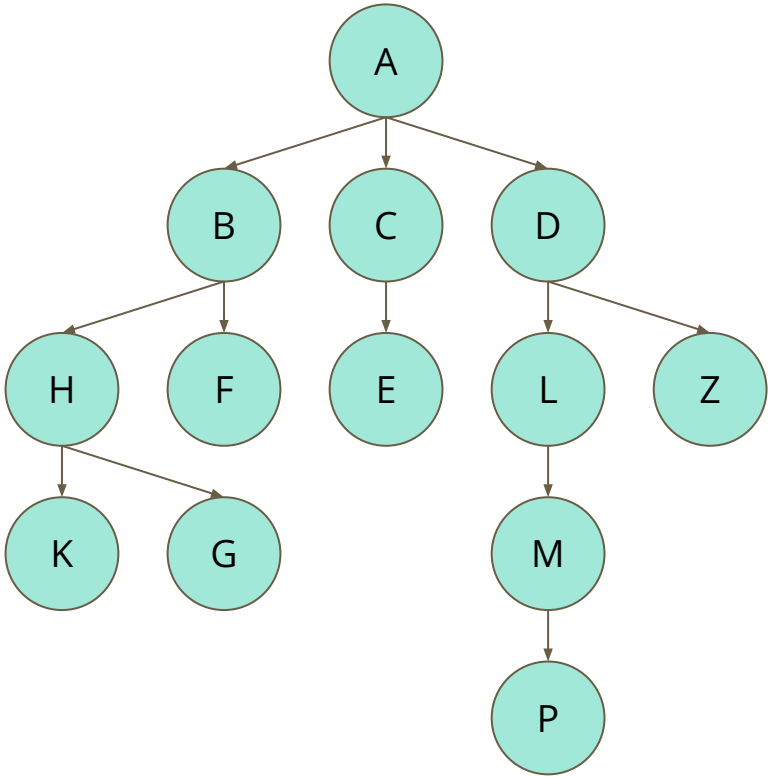
**Ancestro propio(u,v)**: u es ancestro propio de v si u es ancestro de v y u ≠ v. Ej: acentropropio(A,E), ancestropropio(D, P), ancestropropio(D,L)



# TDA ÁRBOL - Definiciones

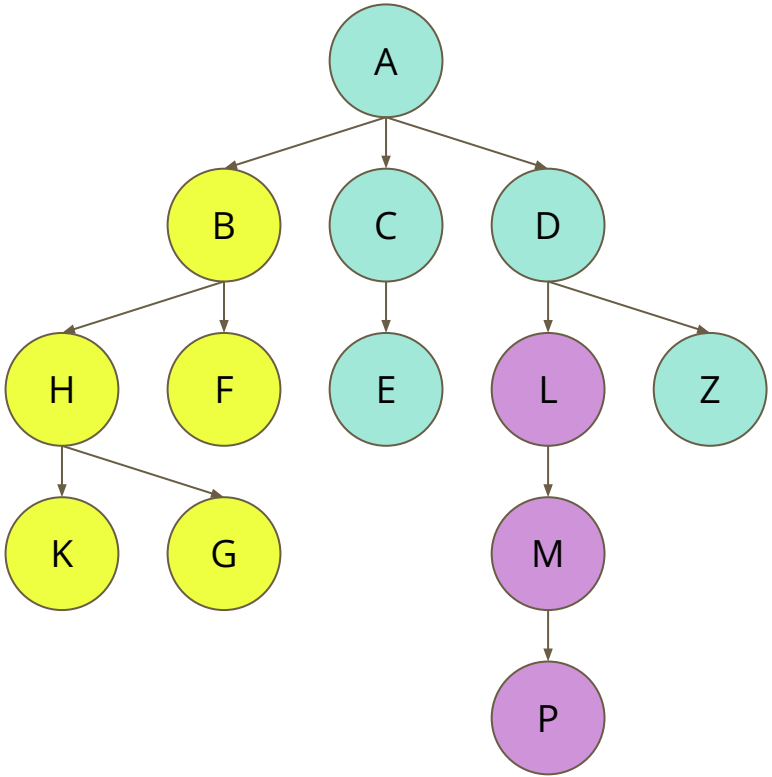
**Descendiente(u,v)**: Un nodo u es descendiente de un nodo v si v es un ancestro de u. Ej: descendiente(C,C), descendiente(E,A), descendiente(P,D)

**Descendiente propio(u,v)**: u es descendiente propio de v si u es descendiente de v y u ≠ v. Ej: descendientepropio(E,A).



# TDA ÁRBOL - Definiciones

**Subárbol**: El subárbol de T con raíz en el nodo v es el árbol consistiendo de todos los descendientes de v. Ej: Mostramos dos subárboles con raíces B y L respectivamente.

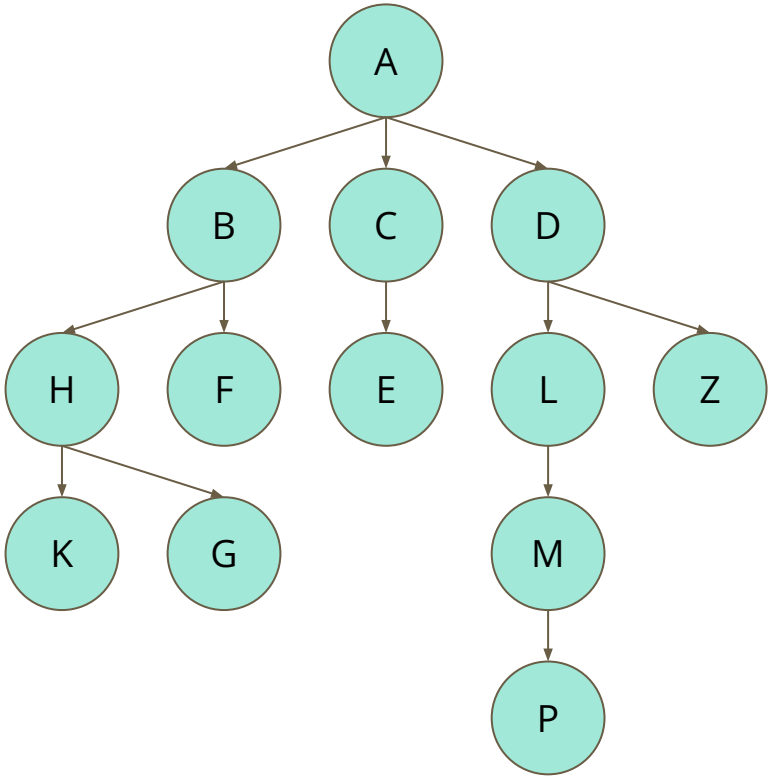


# TDA ÁRBOL - Definiciones

**Arco no dirigido**: Un arco es un par de nodos (u,v) o (v,u) tales que u es padre de v. Ej: (D,Z) y (Z,D) son arcos no dirigidos.

**Camino**: Un camino de T es una secuencia de nodos tales que cualquier par de nodos consecutivos en la secuencia forman un arco.

- Ej: A, B, F es un camino
- Ej: F, B, A, D, L, M es otro camino.

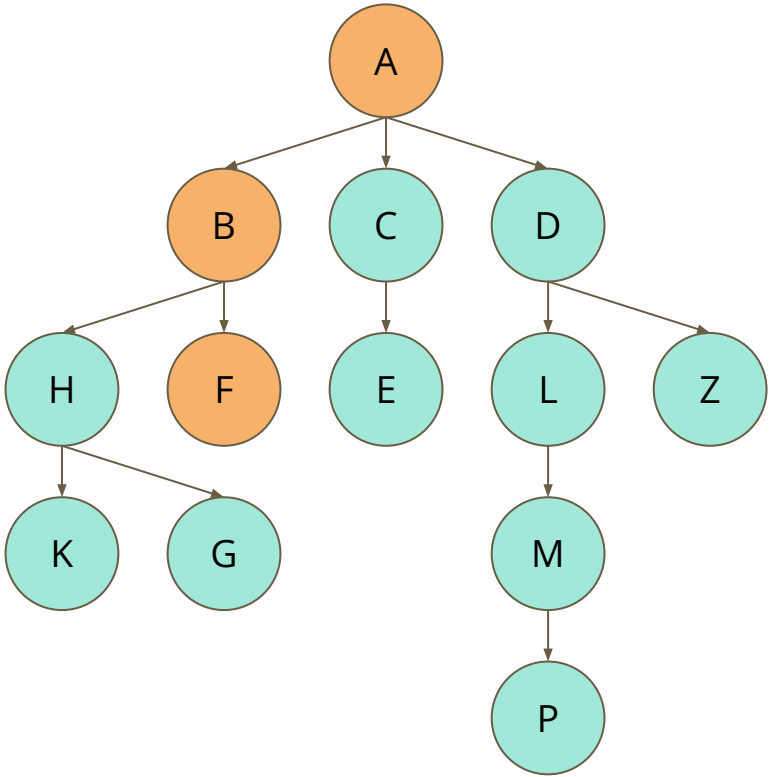


# TDA ÁRBOL - Definiciones

**Arco no dirigido**: Un arco es un par de nodos (u,v) o (v,u) tales que u es padre de v. Ej: (D,Z) y (Z,D) son arcos no dirigidos.

**Camino**: Un camino de T es una secuencia de nodos tales que cualquier par de nodos consecutivos en la secuencia forman un arco.

- Ej: **A, B, F es un camino**
- Ej: F, B, A, D, L, M es otro camino.

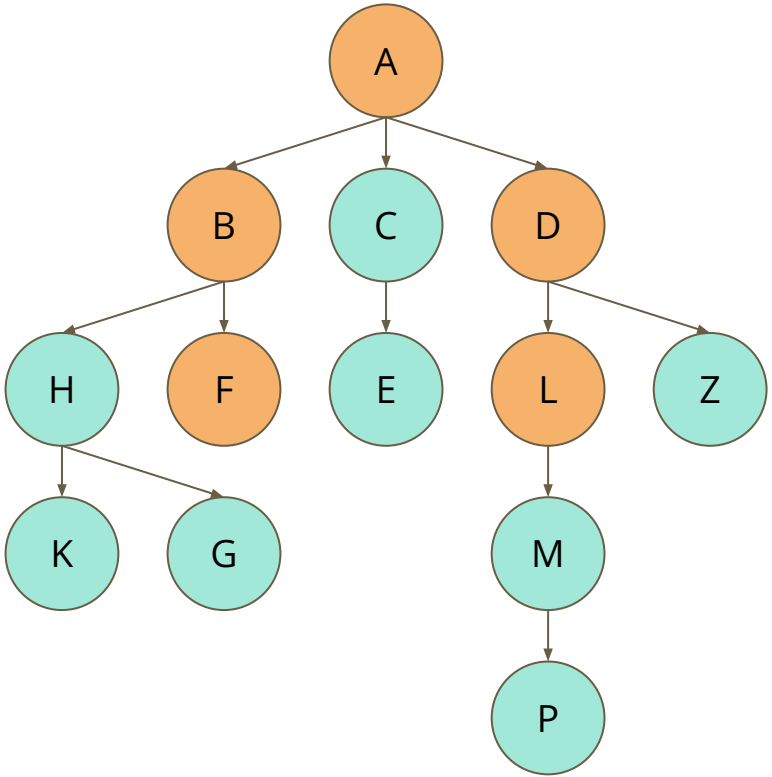


# TDA ÁRBOL - Definiciones

**Arco no dirigido**: Un arco es un par de nodos (u,v) o (v,u) tales que u es padre de v. Ej: (D,Z) y (Z,D) son arcos no dirigidos.

**Camino**: Un camino de T es una secuencia de nodos tales que cualquier par de nodos consecutivos en la secuencia forman un arco.

- Ej: A, B, F es un camino
- Ej: **F, B, A, D, L, M es otro camino.**

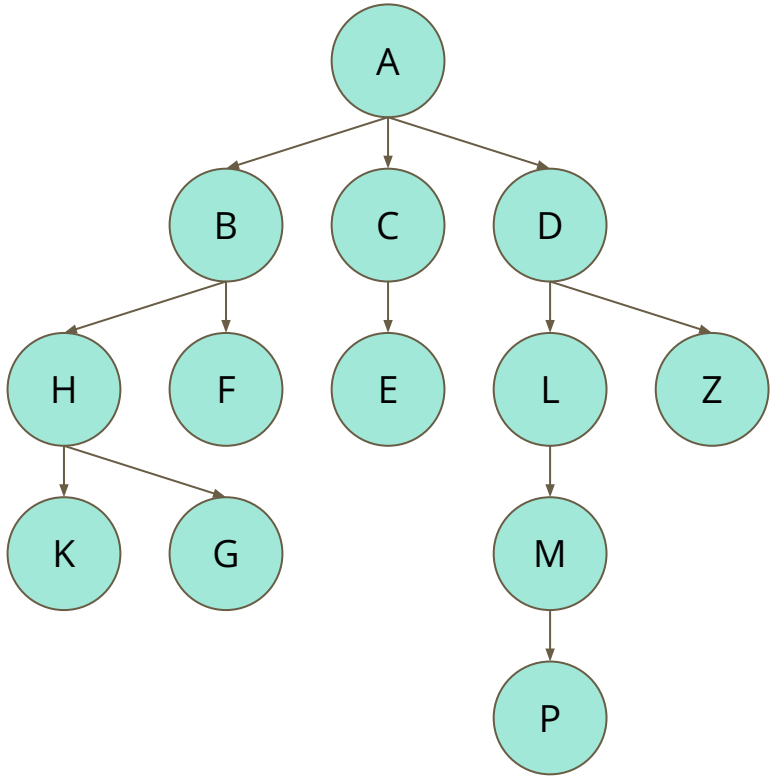


# TDA ÁRBOL - Definiciones

La relación de herencia de clases en Java forman un árbol.

La raíz, java.lang.Object es un ancestro de todas las clases.

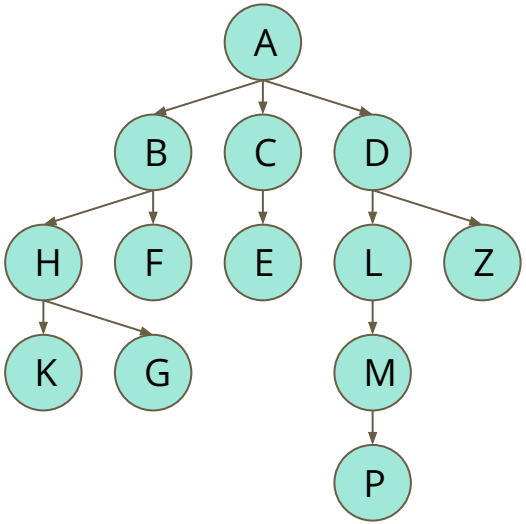
Cada clase C es un descendiente de Object y C es la raíz del subárbol formado por todas las clases descendientes de C.



# TDA ÁRBOL - Definiciones

Un árbol se dice **ordenado** si existe un orden lineal para los hijos de cada nodo. Es decir, se puede identificar el primer hijo, el segundo hijo y así sucesivamente. Tal orden se visualiza de izquierda a derecha de acuerdo a tal ordenamiento.

Por ejemplo, para el árbol de la figura si consideramos el nodo B, podemos decir que H es el primer hijo de B y F es el segundo y último hijo de B.



# TDA ÁRBOL - Definiciones

Position:

**element():** retorna el objeto almacenado en esta posición

https://github.com/ed-dcic-uns/tdas/blob/main/src/main/java/ar/edu/uns/cs/ed/tdas/tdaarbol/Tree.java

# TDA ÁRBOL - Definiciones

Tree: Métodos de acceso (reciben y retornan posiciones)

**root():** Retorna la raíz del árbol, error si el árbol está vacío

**parent(v):** Retorna el padre de v, error si v es la raíz

**children(v):** Retorna una colección iterable conteniendo los hijos del nodo v

# TDA ÁRBOL - Definiciones

Tree: Métodos de acceso (reciben y retornan posiciones)

**root():** Retorna la raíz del árbol, error si el árbol está vacío

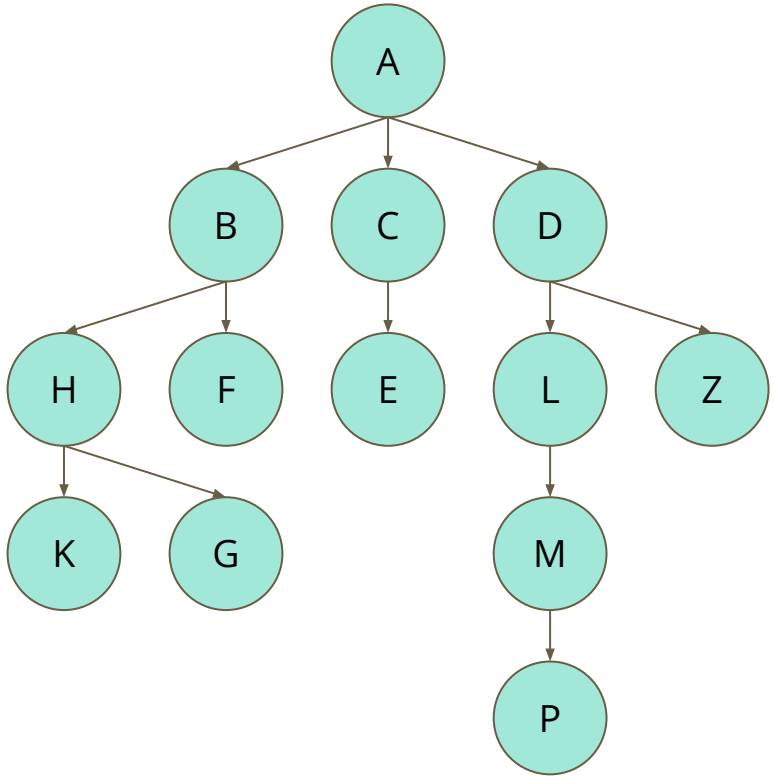
**parent(v):** Retorna el padre de v, error si v es la raíz

**children(v):** Retorna una colección iterable conteniendo los hijos del nodo v

*Nota: En nuestros árboles, children mantiene el orden de izquierda a derecha. Si v es una hoja children(v) está vacía.*

# TDA ÁRBOL - Definiciones

T.root() →pA
T.parent(pF) →pB
T.parent(pM) →pL
T.parent(pA) →error
T.parent(T.parent(pP)) → pL
T.children(pB) →[pH, PF]
T.children(pA) →[pB, pC, pD]
T.children(pE) →[]
T.children(pG) →[]
pG.element() →‘G’
pA.element() →‘A’
T.root().element() →‘A’
T.children(T.root()).iterator().next().element() →‘B’



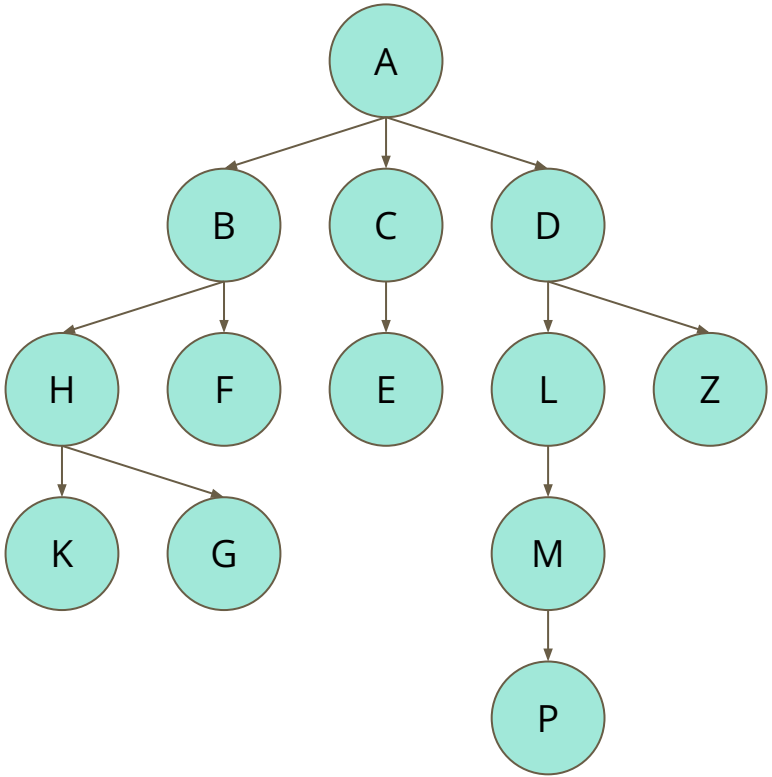
# TDA ÁRBOL - Definiciones

Tree: Métodos de consulta

**isInternal(v)**: Testea si v es un nodo interno

**isExternal(v):** Testea si v es una hoja

**isRoot(v):** Testea si v es la raíz



# TDA ÁRBOL - Definiciones

T.isInternal(T.root()) →true

T.isInternal(pG) →false

T.isInternal(pH) →true

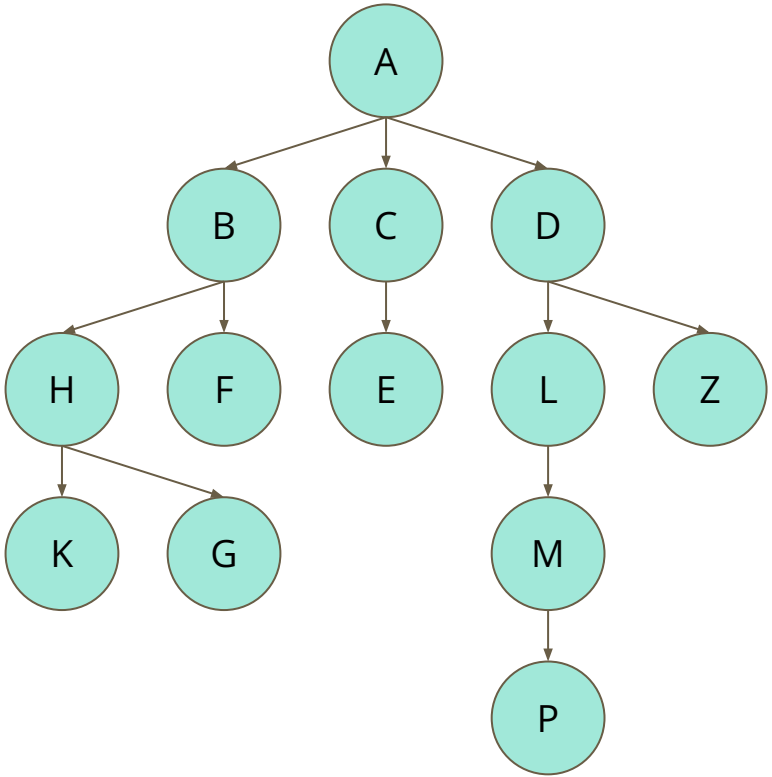
T.isExternal(T.root()) →false

T.isExternal(pG) →true

T.isExternal(pH) →false

T.isRoot(pG) →false

T.isRoot(T.root()) →true



# TDA ÁRBOL - Definiciones

Tree: Métodos genéricos

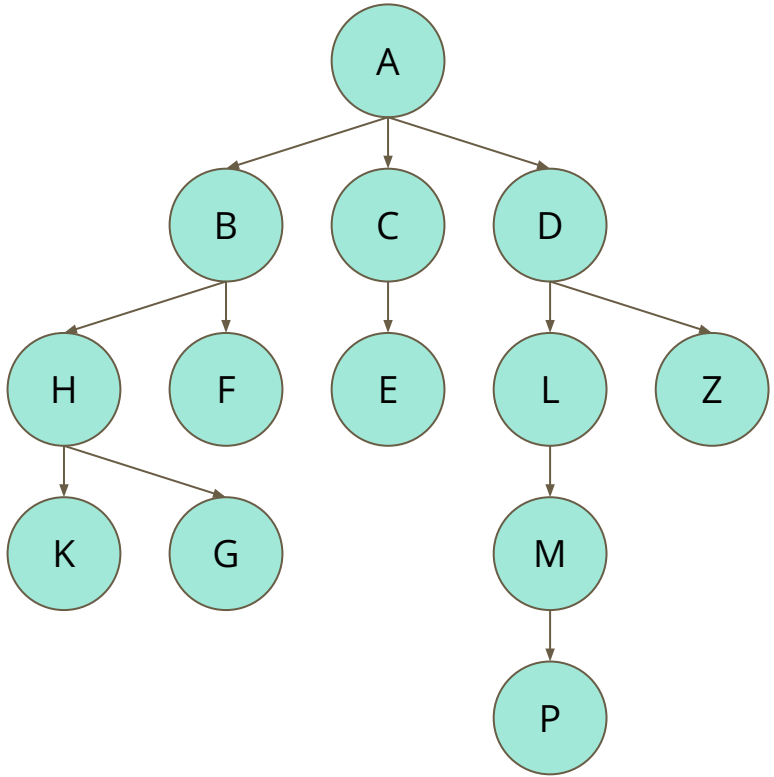
**size():** Retorna el número de nodos del árbol

**isEmpty():** Testea si el árbol tiene o no nodos

**iterator():** Retorna un iterador con los elementos ubicados en los nodos del árbol

**positions():** Retorna una colección iterable de los nodos del árbol

**replace(v,e):** Reemplaza el elemento almacenado en la posición v por el elemento e pasado por parámetro. Devuelve el elemento reemplazado.



# TDA ÁRBOL - Definiciones

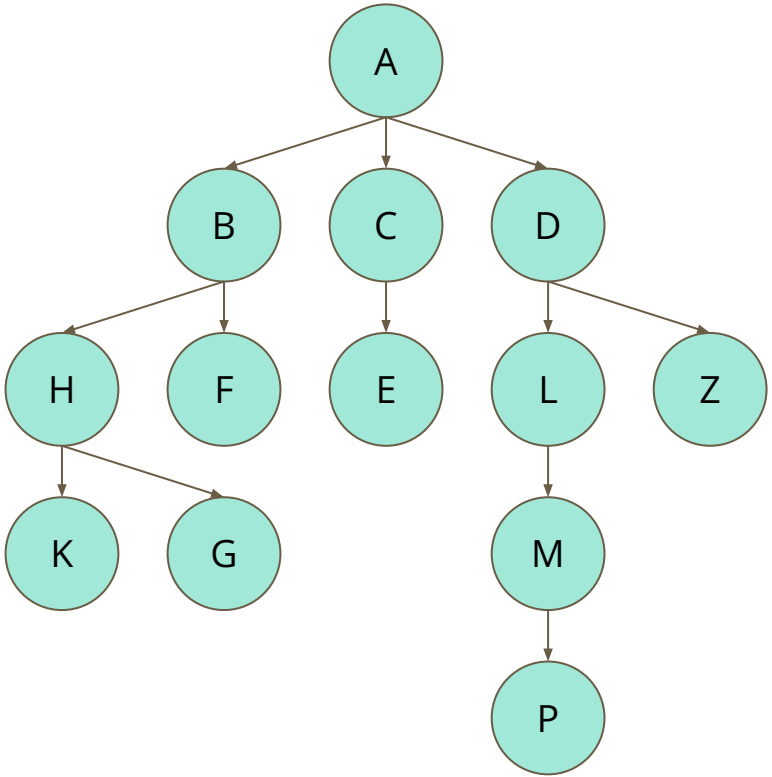
T.size() →13

T.isEmpty() → false

Imprimir rótulos del árbol T:
*for( Character c : T ) { .... }*

Imprimir rótulos del árbol T:
*Iterator<Character> it = T.iterator();*
*while( it.hasNext() ) { char c = it.next(); ... }*

Imprimir rótulos del árbol T a partir de sus posiciones:
*for( Position<Character> p : T.positions() )*
*{ .... p.element() .... }*



# TDA ÁRBOL - Definiciones

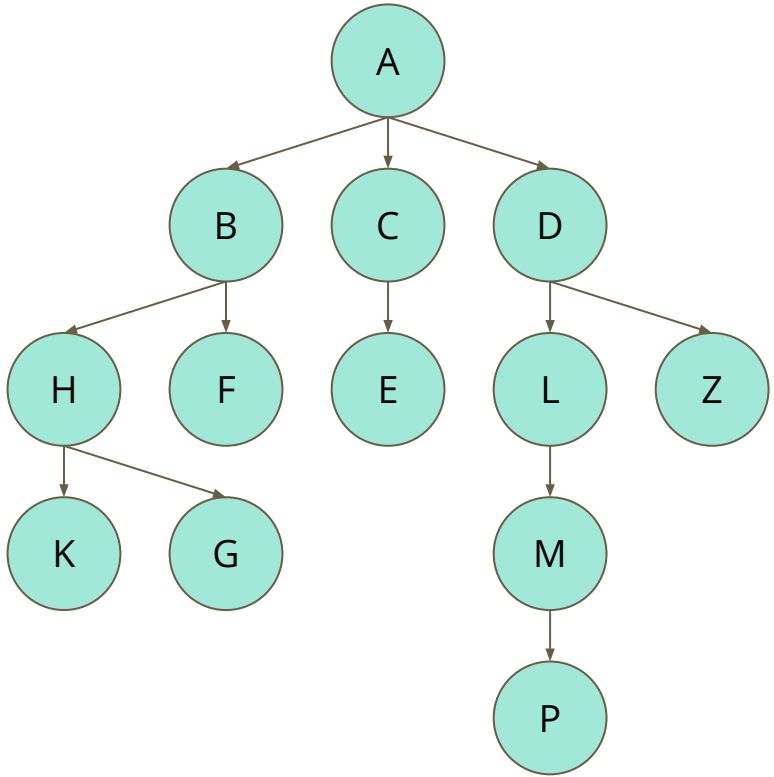
T.size() →13

T.isEmpty() → false

**Imprimir rótulos del árbol T:**
***for( Character c : T ) { .... }***

Imprimir rótulos del árbol T:
*Iterator<Character> it = T.iterator();*
*while( it.hasNext() ) { char c = it.next(); ... }*

Imprimir rótulos del árbol T:
*for( Position<Character> p : T.positions() )*
*{ .... p.element() .... }*



# TDA ÁRBOL - Definiciones

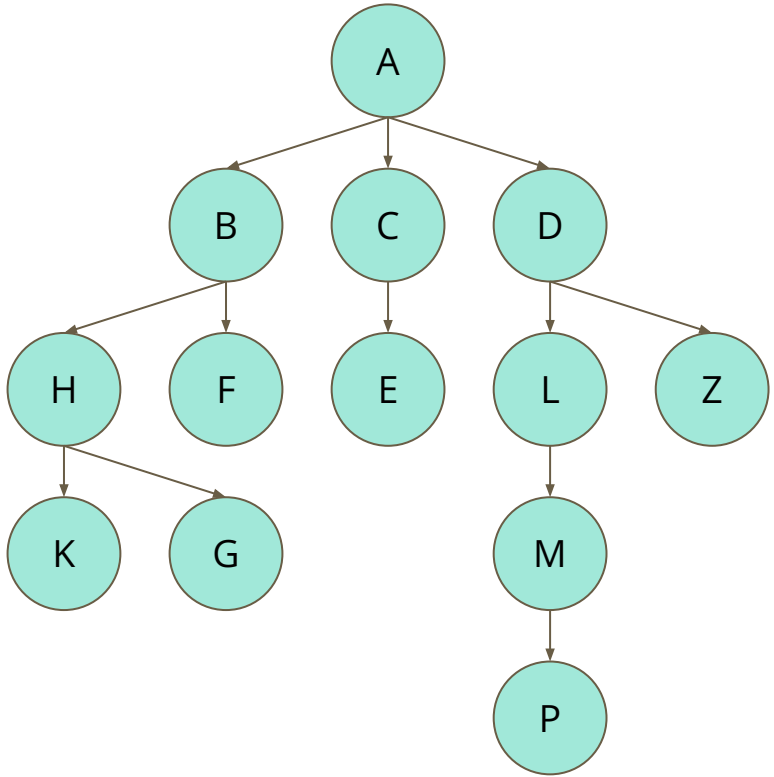
T.size() →13

T.isEmpty() → false

Imprimir rótulos del árbol T:
*for( Character c : T ) { .... }*

**Imprimir rótulos del árbol T:**
***Iterator<Character> it = T.iterator();***
***while( it.hasNext() ) { char c = it.next(); ... }***

Imprimir rótulos del árbol T:
*for( Position<Character> p : T.positions() )*
*{ .... p.element() .... }*



# TDA ÁRBOL - Definiciones

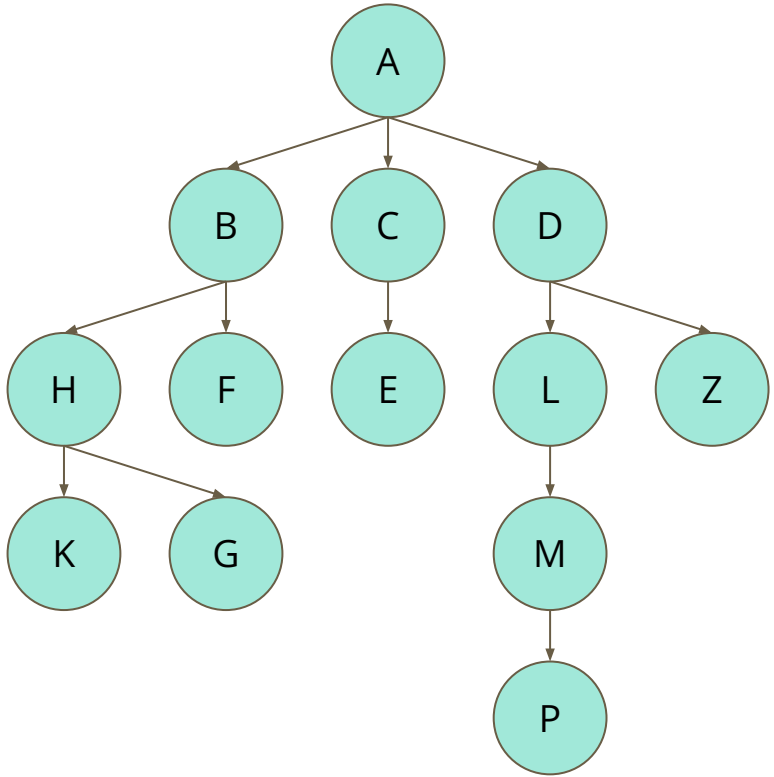
T.size() →13

T.isEmpty() → false

Imprimir rótulos del árbol T:
*for( Character c : T ) { .... }*

Imprimir rótulos del árbol T:
*Iterator<Character> it = T.iterator();*
*while( it.hasNext() ) { char c = it.next(); ... }*

**Imprimir rótulos del árbol T a partir de sus posiciones:**
***for( Position<Character> p : T.positions() )***
***{ .... p.element() .... }***



# TDA ÁRBOL - Definiciones

Tree: Métodos de modificación (agregados por la cátedra a [GT])

**createRoot(e):** crea un nodo raíz con rótulo e

**addFirstChild(p, e):** agrega un primer hijo al nodo p con rótulo e

**addLastChild(p, e):** agrega un último hijo al nodo p con rótulo e

**addBefore(p, rb, e):** Agrega un nodo con rótulo e como hijo de un nodo padre p dado. El nuevo nodo se agregará delante de otro nodo hermano rb también dado.

**addAfter(p, lb, e):** Agrega un nodo con rótulo e como hijo de un nodo padre p dado. El nuevo nodo se agregará detrás de otro hermano lb también dado.

# TDA ÁRBOL - Definiciones

Tree: Métodos de modificación (agregados por la cátedra a [GT])

**removeExternalNode (p):** Elimina la hoja p

**removeInternalNode (p):** Elimina el nodo interno p. Los hijos del nodo eliminado lo reemplazan en el mismo orden en el que aparecen. La raíz se puede eliminar si tiene un único hijo.

**removeNode (p):** Elimina el nodo p.

# TDA ÁRBOL - Interface

```
public interface Tree<E> extends Iterable<E>
{
public int size();
public boolean isEmpty();
public Iterator<E> iterator();
public Iterable<Position<E>> positions();
public E replace(Position<E> v, E e)
public Position<E> root()
public Position<E> parent(Position<E> v)
public Iterable<Position<E>> children(Position<E> v)
public boolean isInternal(Position<E> v)
public boolean isExternal(Position<E> v)
public boolean isRoot(Position<E> v)
public void createRoot(E e)
public Position<E> addFirstChild(Position<E> p, E e)
public Position<E> addLastChild(Position<E> p, E e)
public Position<E> addBefore(Position<E> p, Position<E> rb, E e)
public Position<E> addAfter (Position<E> p, Position<E> lb, E e)
public void removeExternalNode (Position<E> p)
public void removeInternalNode (Position<E> p)
public void removeNode (Position<E> p)}
```

# TDA ÁRBOL - Interfaz

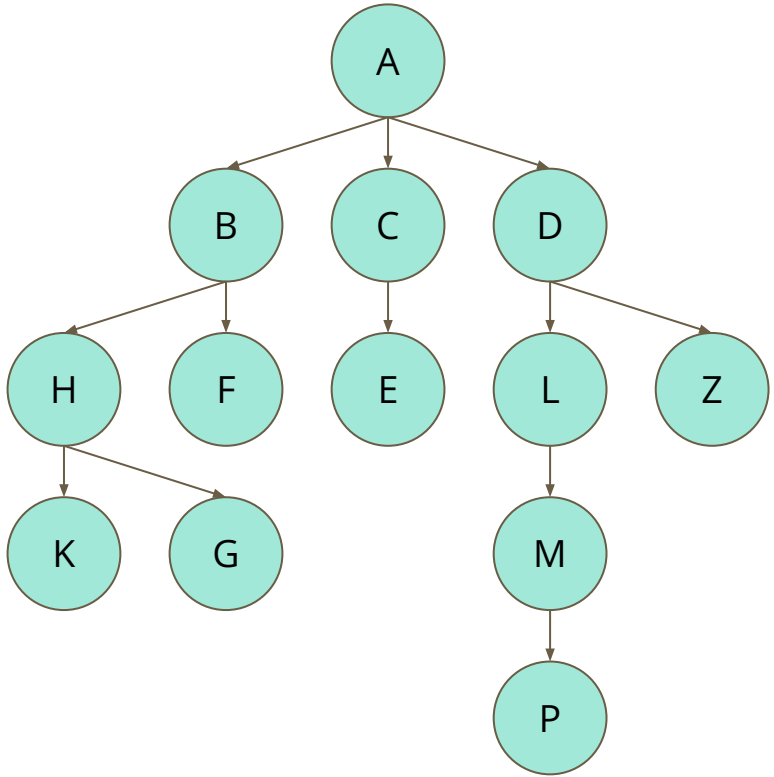
```
Tree<Character> arbol = new Arbol<Character>();
// Creo un árbol de caracteres
arbol.createRoot( 'A' ); // Agrego la raíz
Position<Character> raiz = arbol.root();

// Agrego hijos de A:
Position<Character> pB = arbol.addLastChild( raiz, 'B' );
Position<Character> pC = arbol.addLastChild( raiz, 'C' );
Position<Character> pD = arbol.addLastChild( raiz, 'D' );

// Agrego hijos de B:
Position<Character> pH = arbol.addLastChild( pB, 'H' );
arbol.addLastChild( pB, 'F' );

// Agrego los hijos de H:
arbol.addFirstChild( pH, 'G' );
arbol.addFirstChild( pH, 'K' );

// Agrego hijo de C:
arbol.addLastChild( pC, 'E' );
// Completar con descendientes propios de D
```



# TDA ÁRBOL - Interfaz

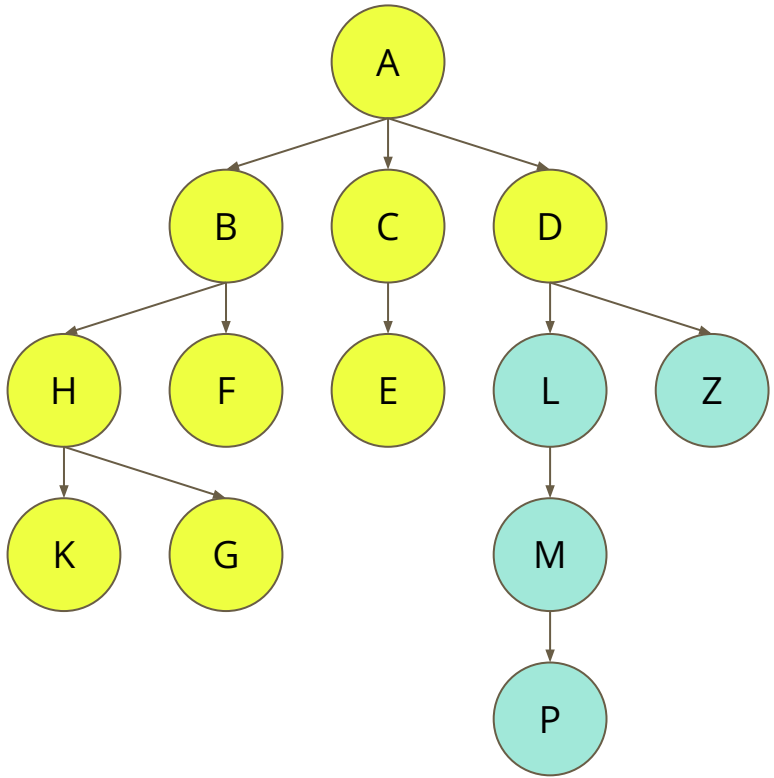
```
Tree<Character> arbol = new Arbol<Character>();
// Creo un árbol de caracteres
arbol.createRoot( 'A' ); // Agrego la raíz
Position<Character> raiz = arbol.root();

// Agrego hijos de A:
Position<Character> pB = arbol.addLastChild( raiz, 'B' );
Position<Character> pC = arbol.addLastChild( raiz, 'C' );
Position<Character> pD = arbol.addLastChild( raiz, 'D' );

// Agrego hijos de B:
Position<Character> pH = arbol.addLastChild( pB, 'H' );
arbol.addLastChild( pB, 'F' );

// Agrego los hijos de H:
arbol.addFirstChild( pH, 'G' );
arbol.addFirstChild( pH, 'K' );

// Agrego hijo de C:
arbol.addLastChild( pC, 'E' );
// Completar con descendientes propios de D
```



# TDA ÁRBOL - Definiciones

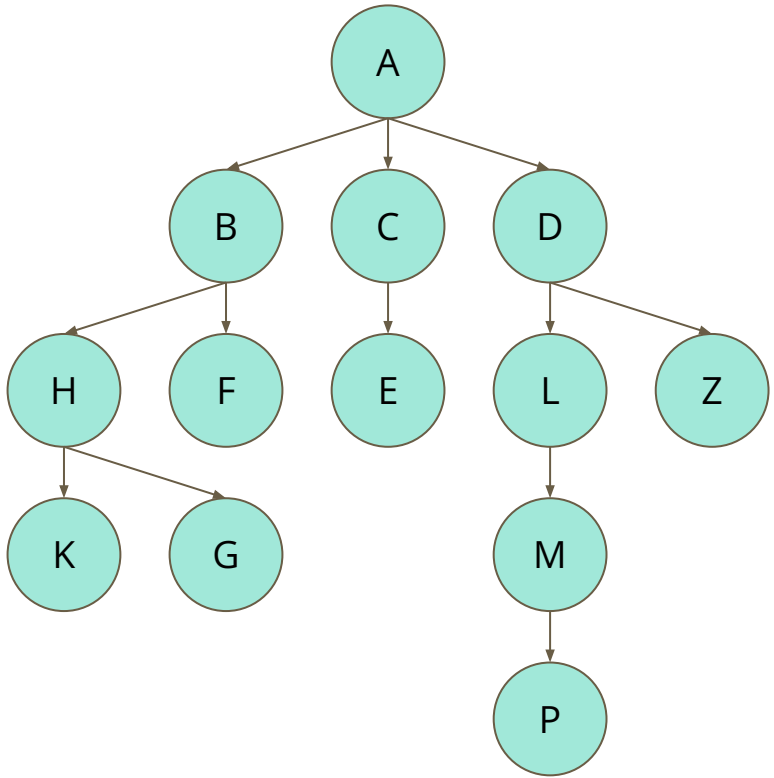
Profundidad de un nodo v en un árbol T: Longitud del camino de la raíz de T al nodo v = cantidad de ancestros propios de v

Longitud de un camino: Cantidad de arcos del camino

Altura de un nodo v en un árbol T: Longitud del camino más largo a una hoja en el subárbol con raíz v.

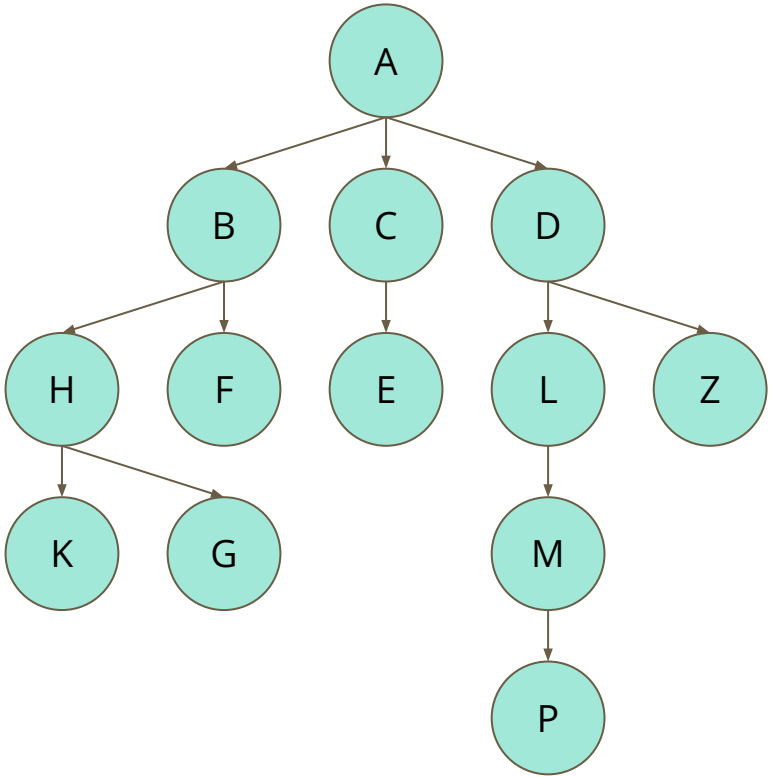
Altura de un árbol T: Altura del nodo raíz de T.

profundidad de A = 0
profundidad de E = 2
profundidad de P = 4
Altura de B = 2
Altura de D = 3
Altura de G = 0
Profundidad de G = 3



# TDA ÁRBOL

La naturaleza recursiva de la definición de árboles invita a plantear soluciones recursivas a muchos de sus problemas.



# TDA ÁRBOL

Profundidad de un nodo v en un árbol T:

Longitud del camino de la raíz de T al nodo v = cantidad de ancestros propios de v

Profundidad(T,v) = Longitud del camino de v a T.root()

# TDA ÁRBOL

Profundidad de un nodo v en un árbol T:

Algoritmo profundidad( T, v )

Caso Base: Si v es la raíz de T,

profundidad(T,v) = 0

Caso Recursivo: Si v no es la raíz de T,

profundidad(T,v) = 1 + profundidad( T, T.parent(v))

# TDA ÁRBOL

Profundidad de un nodo v en un árbol T:

```
public static <E> int profundidad( Tree<E> T, Position<E> v ) {
        if (T.isRoot(v) )
                return 0;
        else
                return 1 + profundidad( T, T.parent( v ) );
}
```

# TDA ÁRBOL

Altura de un nodo v en un árbol T:

Longitud del camino más largo a una hoja en el subárbol con raíz v.

Proposición:

> La altura de un árbol T no vacío es la máxima profundidad de las hojas de T

# TDA ÁRBOL

La altura de un árbol T no vacío es la máxima profundidad de las hojas de T

Algoritmo Altura(T)

```
h ← 0
para cada vértice v en T hacer
    si v es una hoja en T entonces
        h ← max( h, profundidad( T, v ) )
retornar h
```

# TDA ÁRBOL

La altura de un árbol T no vacío es la máxima profundidad de las hojas de T

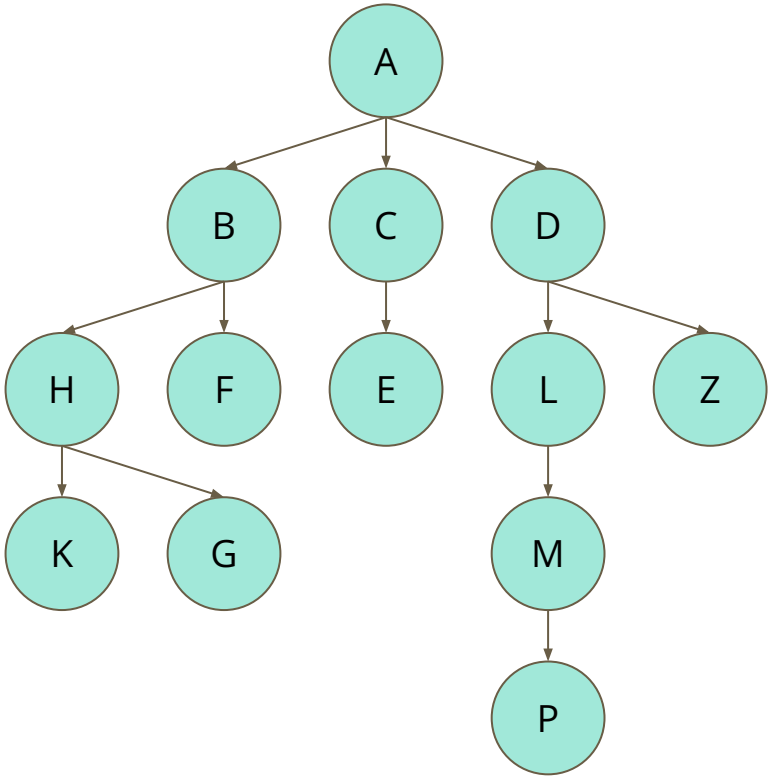
Implementación Java:

```
public static <E> int altura( Tree<E> T ) {
        int h = 0;
        for( Position<E> v : T.positions() )
                if( T.isExternal(v) )
                        h = Math.max( h, profundidad( T, v ) );
        return h;
}
```

# Break.

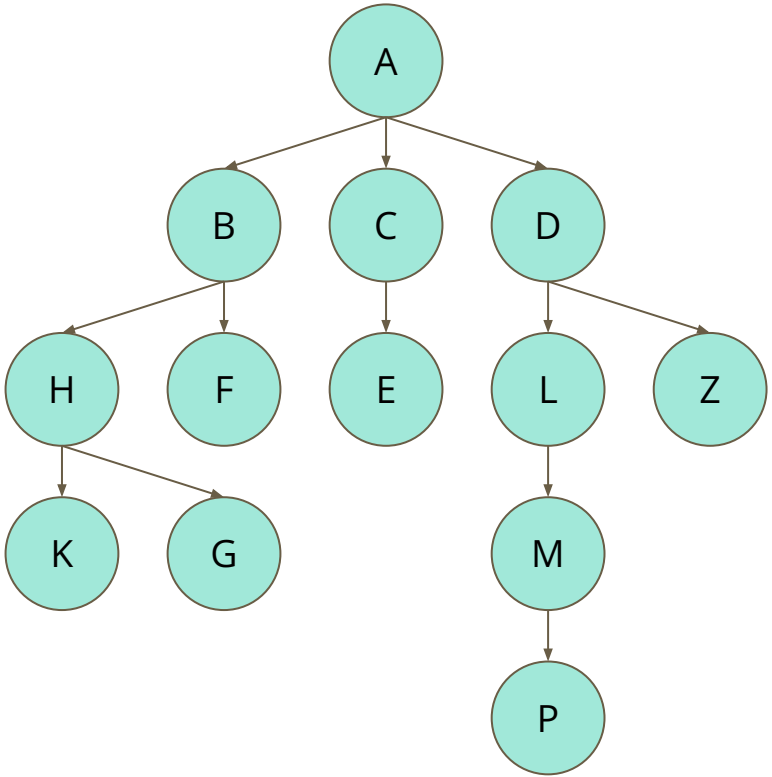
# TDA ÁRBOL

Empiecen a nombrar los rótulos de este árbol. ¿Qué orden siguen?



# TDA ÁRBOL - Recorridos

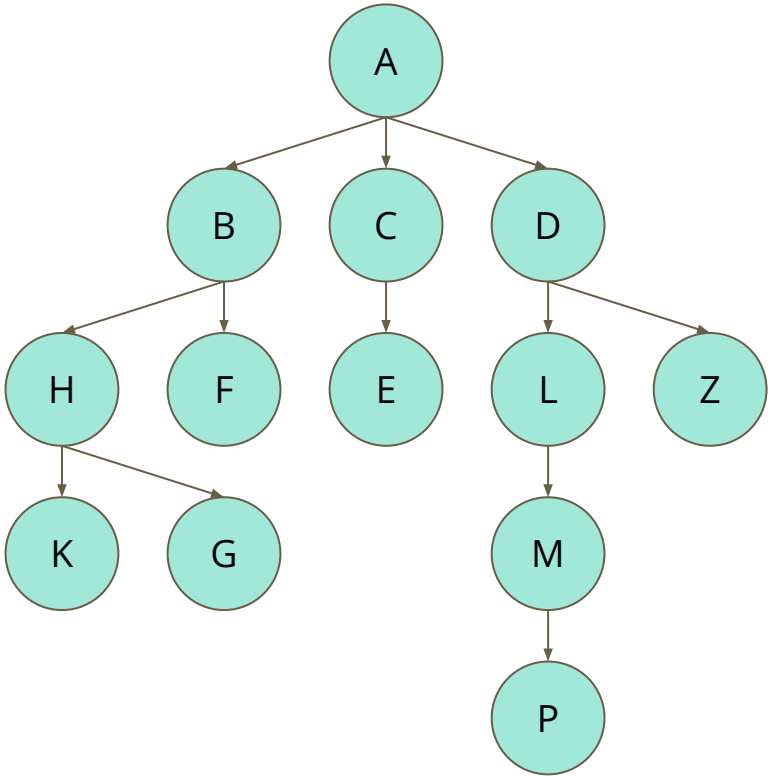
Un recorrido de un árbol T es una manera sistemática de visitar todos los nodos de T



# TDA ÁRBOL - Recorridos

Un recorrido de un árbol T es una manera sistemática de visitar todos los nodos de T

La literatura describe 4 formas de recorrer un árbol:

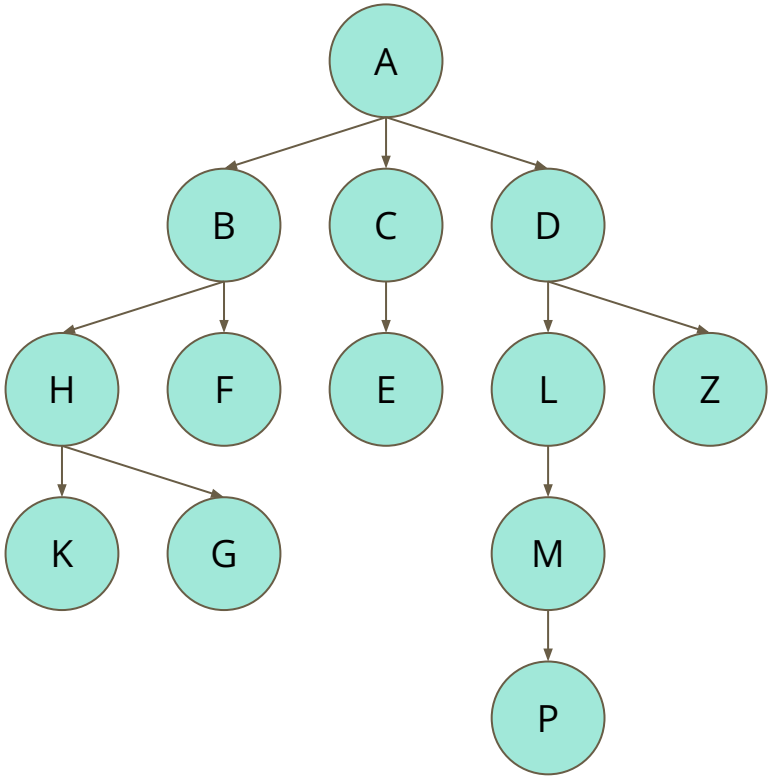


# TDA ÁRBOL - Recorridos

Un recorrido de un árbol T es una manera sistemática de visitar todos los nodos de T

La literatura describe 4 formas de recorrer un árbol:

- Preorden
- Postorden
- Inorder
- Por Niveles

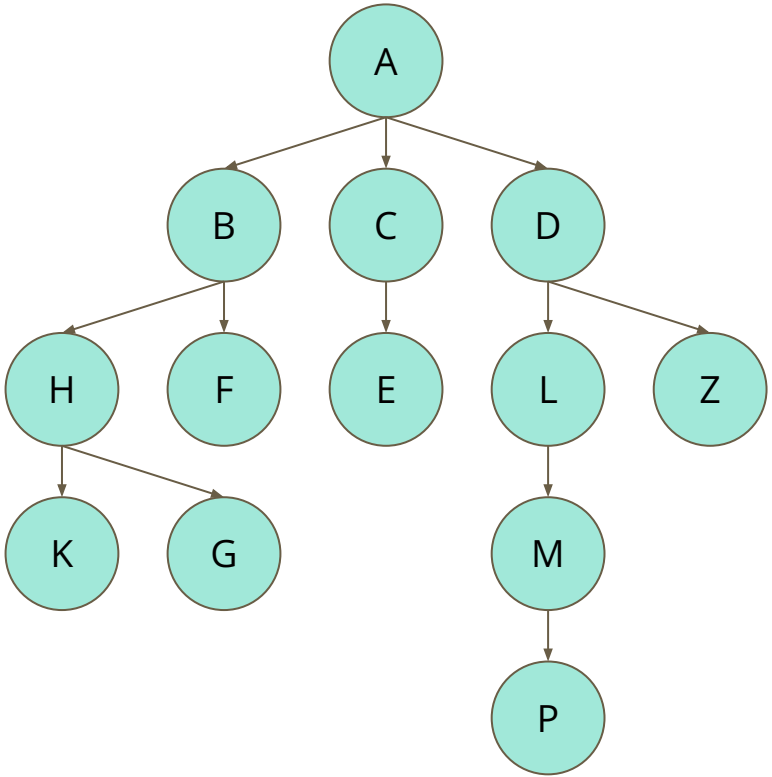


# TDA ÁRBOL - Recorridos

Un recorrido de un árbol T es una manera sistemática de visitar todos los nodos de T

La literatura describe 4 formas de recorrer un árbol:

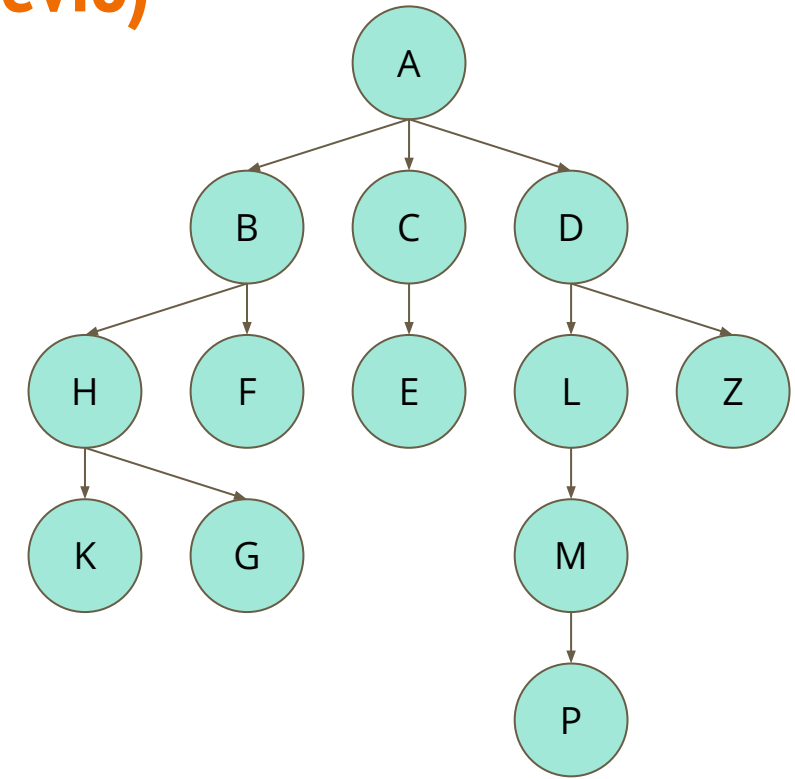
- Preorden (orden previo)
- Postorden (orden posterior)
- Inorder (orden simétrico)
- Por Niveles (level ordering)



# TDA ÁRBOL - Preorden (orden previo)

En el recorrido preorden de un árbol T, la raíz r de T se visita primero y luego se visitan recursivamente cada uno de los subárboles de r.

El algoritmo se llama con **preorden( T, T.root() )**



# TDA ÁRBOL - Preorden (orden previo)

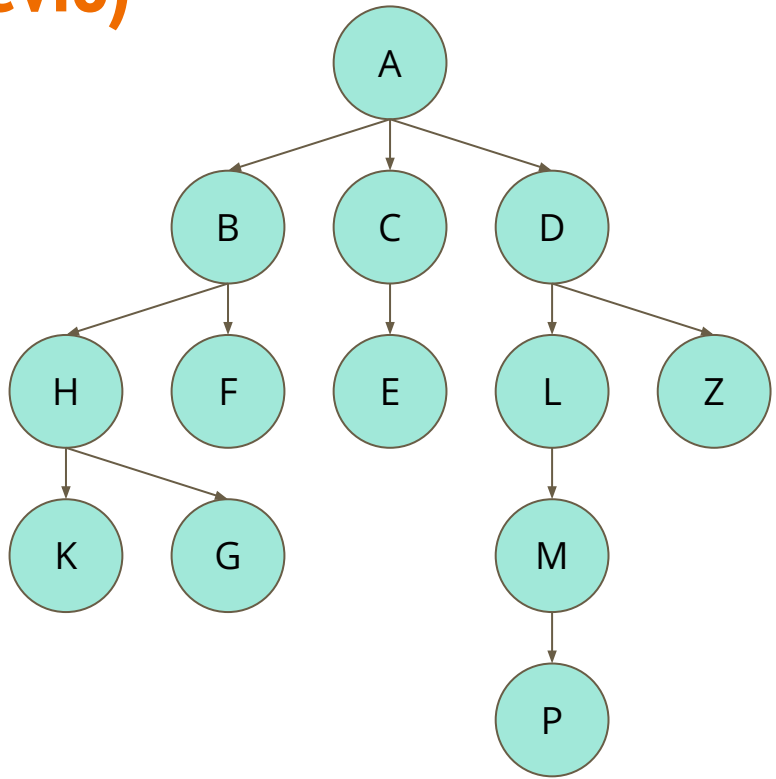
Algoritmo **preorden( T, v )**

Visitar(T, v)

Para cada hijo w de v en T hacer

preorden( T, w )

“Visitar(T,v)” dependerá del problema.

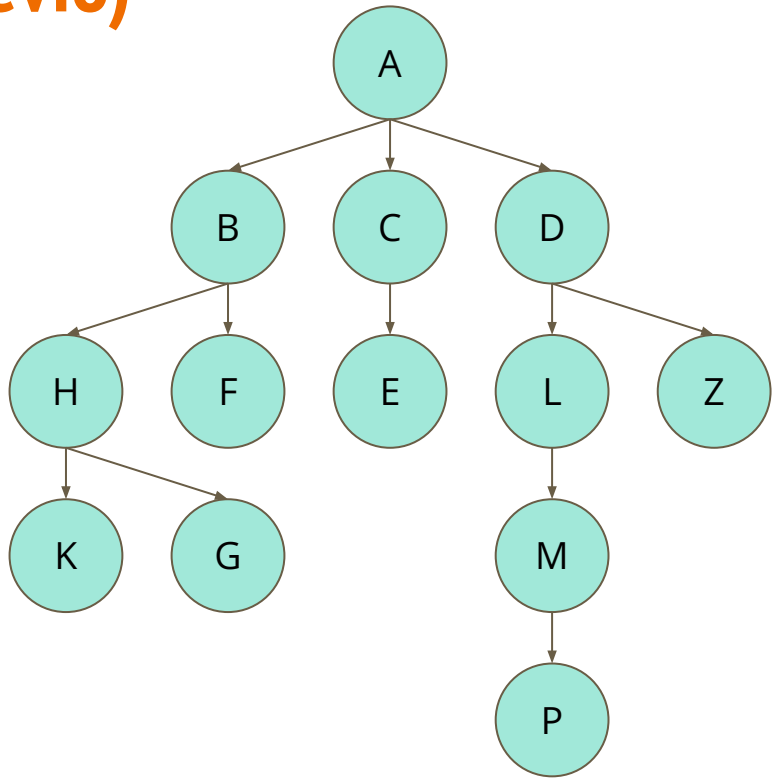


# TDA ÁRBOL - Preorden (orden previo)

Algoritmo **preorden( T, v )**

```
    Visitar(T, v)
    Para cada hijo w de v en T hacer
        preorden( T, w )
```

preorden: A, B, H, K, G, F, C, E, D, L, M, P, Z



# TDA ÁRBOL - Preorden (orden previo)

```
public static <E> String

        toStringPreorder( Tree<E> T, Position<E> v) {

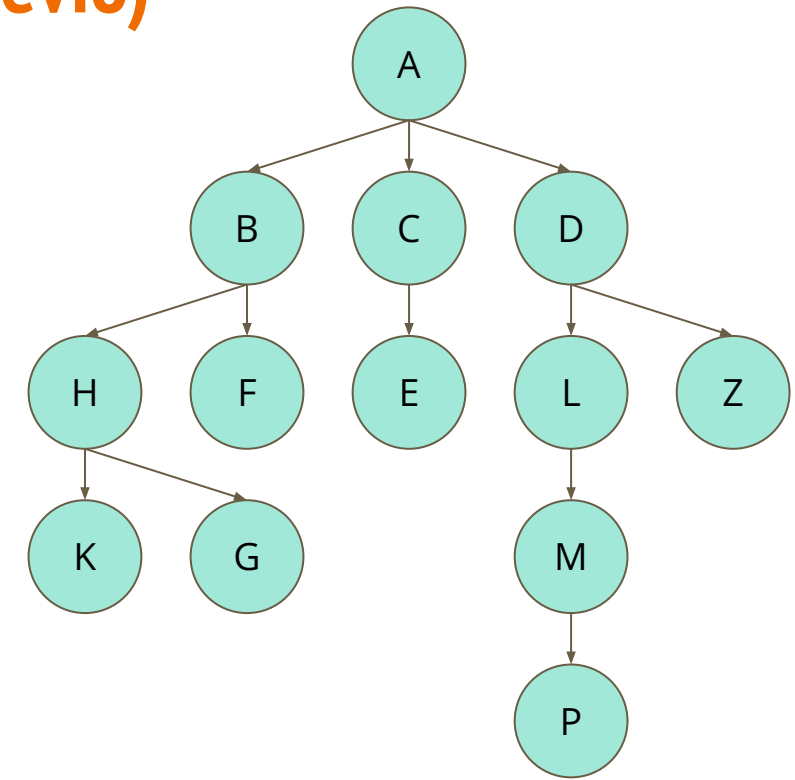
                String s = v.element().toString();

                for( Position<E> w : T.children(v) )

                        s += " – " + toStringPreorder( T, w );

                return s;

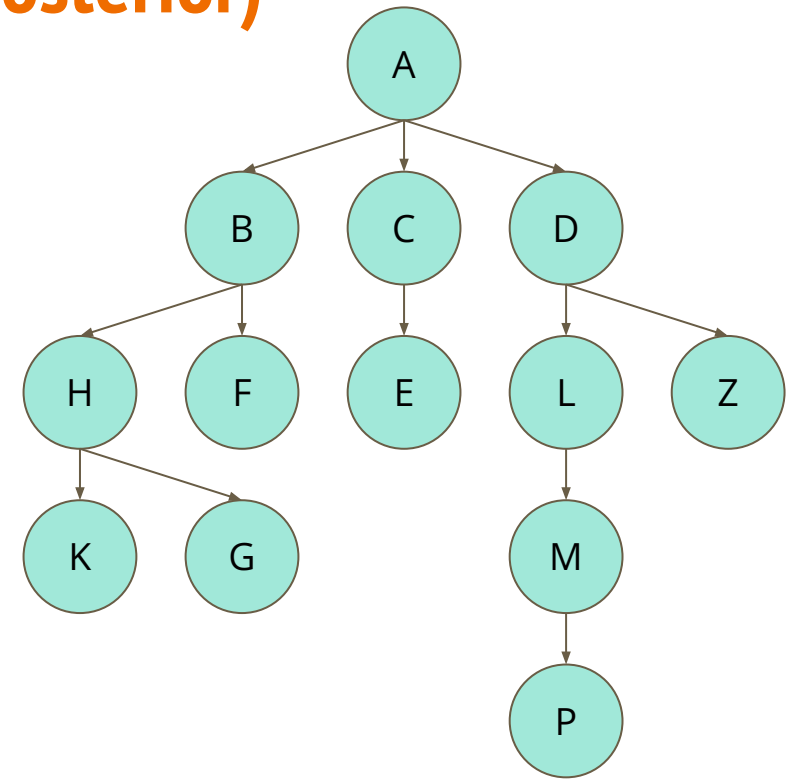
}
```



# TDA ÁRBOL - Postorden (orden posterior)

En el recorrido postorden de un árbol T, la raíz r de T se visita luego de visitar recursivamente cada uno de los subárboles de r.

El algoritmo se llama con **postorden( T, T.root() )**



# TDA ÁRBOL - Postorden (orden posterior)

Algoritmo **postorden( T, v )**

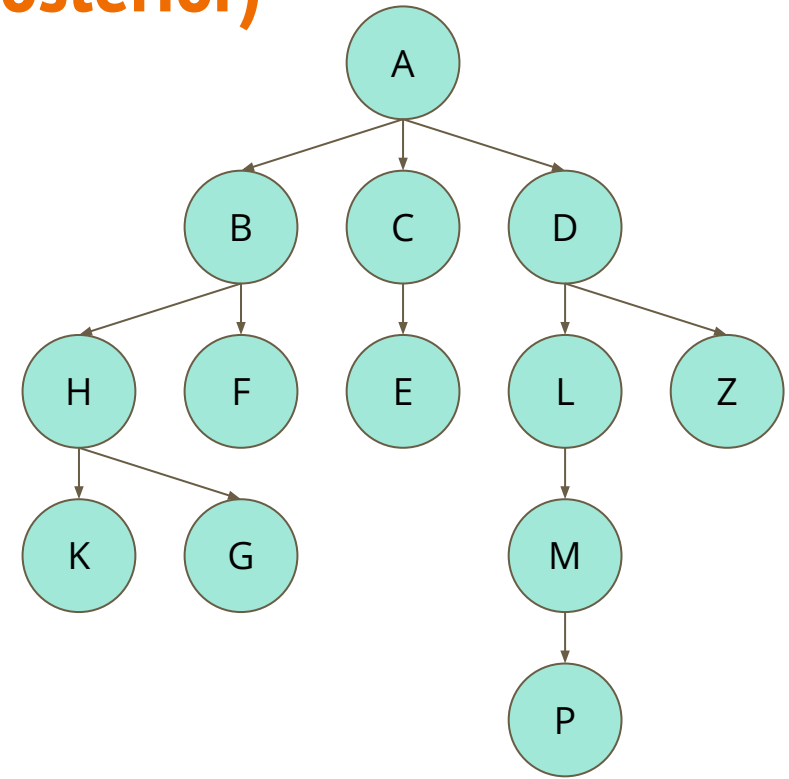
Para cada hijo w de v en T hacer

postorden( T, w )

Visitar(T, v)

“Visitar(T,v)” dependerá del problema.

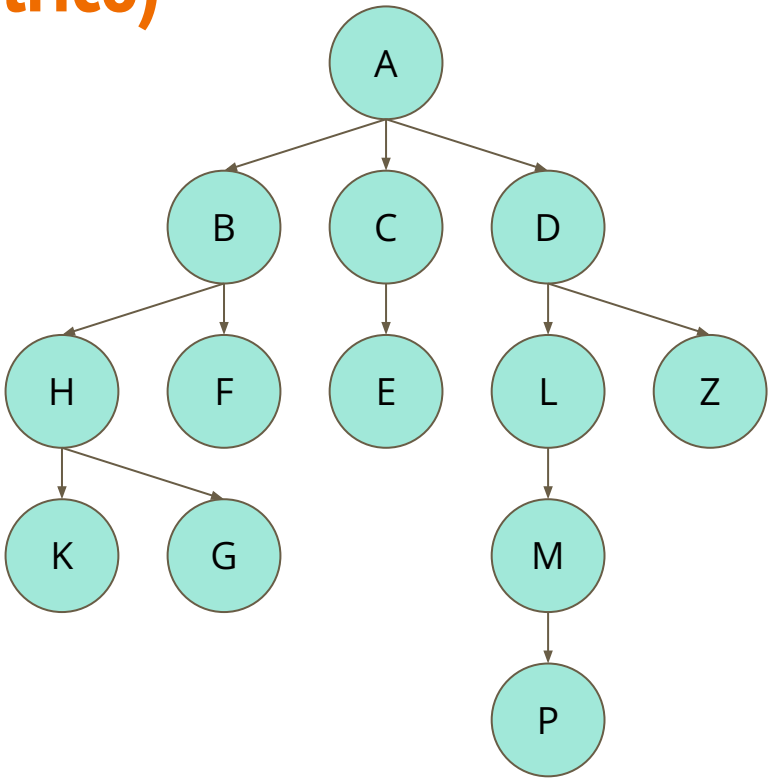
K, G, H, F, B, E, C, P, M, L, Z, D, A



# TDA ÁRBOL - Inorder (orden simétrico)

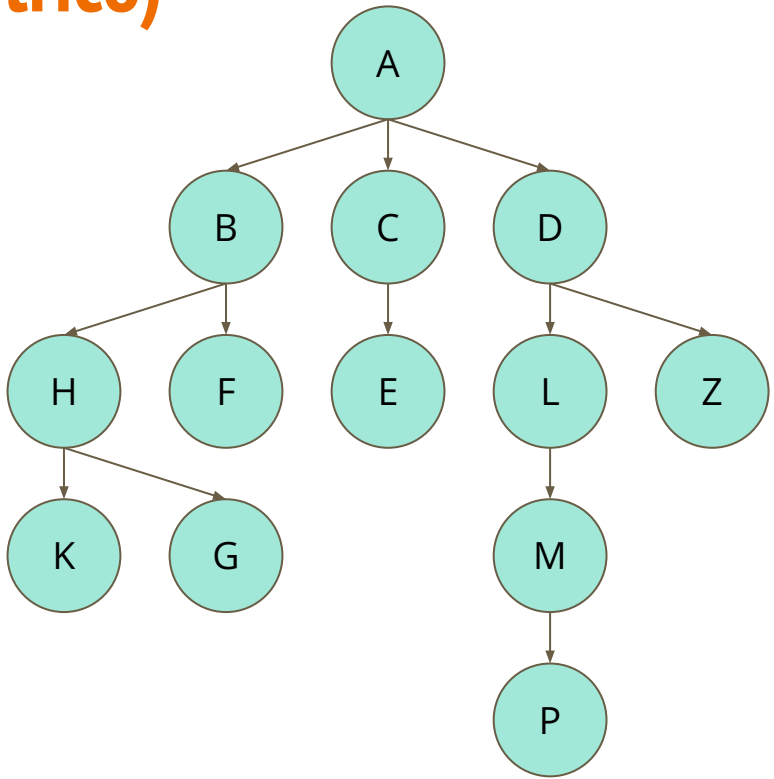
En el recorrido inorder (o simétrico) de un árbol T con raíz r, primero se recorre recursivamente el primer hijo de la raíz r, luego se visita la raíz y luego se visita recursivamente al resto de los hijos de r.

El algoritmo se llama con **inorden( T, T.root() )**



# TDA ÁRBOL - Inorder (orden simétrico)

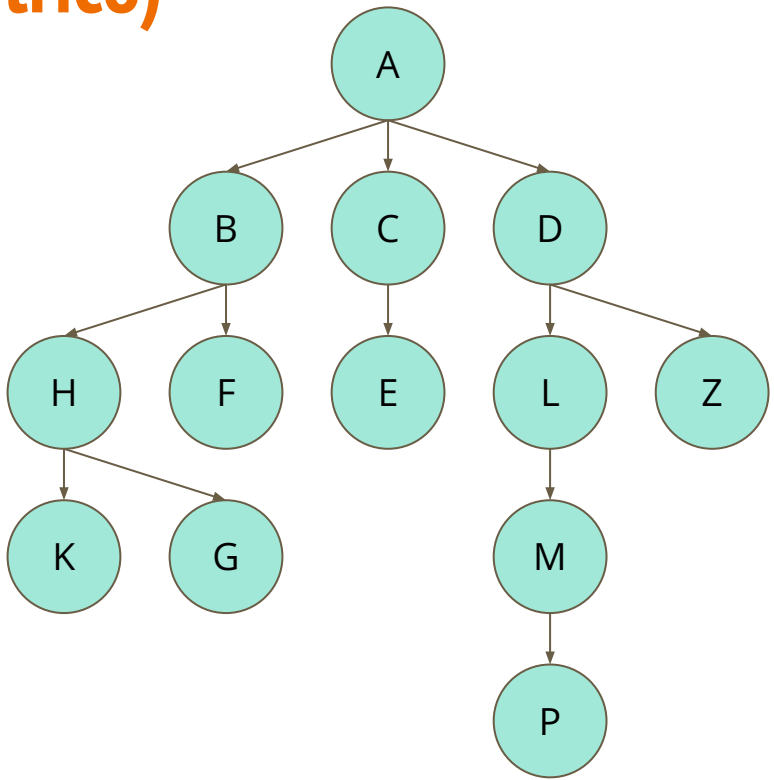
```
Algoritmo inorden( T, v )
    Si v es hoja en T entonces
        Visitar( T, v )
    Sino
        w ← primer hijo de v en T
        inorden( T, w )
        Visitar(T, v)
        mientras w tiene hermano en T hacer
            w ← hermano de w en T
            inorden( T, w )
```



# TDA ÁRBOL - Inorder (orden simétrico)

```
Algoritmo inorden( T, v )
    Si v es hoja en T entonces
        Visitar( T, v )
    Sino
        w ← primer hijo de v en T
        inorden( T, w )
        Visitar(T, v)
        mientras w tiene hermano en T hacer
            w ← hermano de w en T
            inorden( T, w )
```

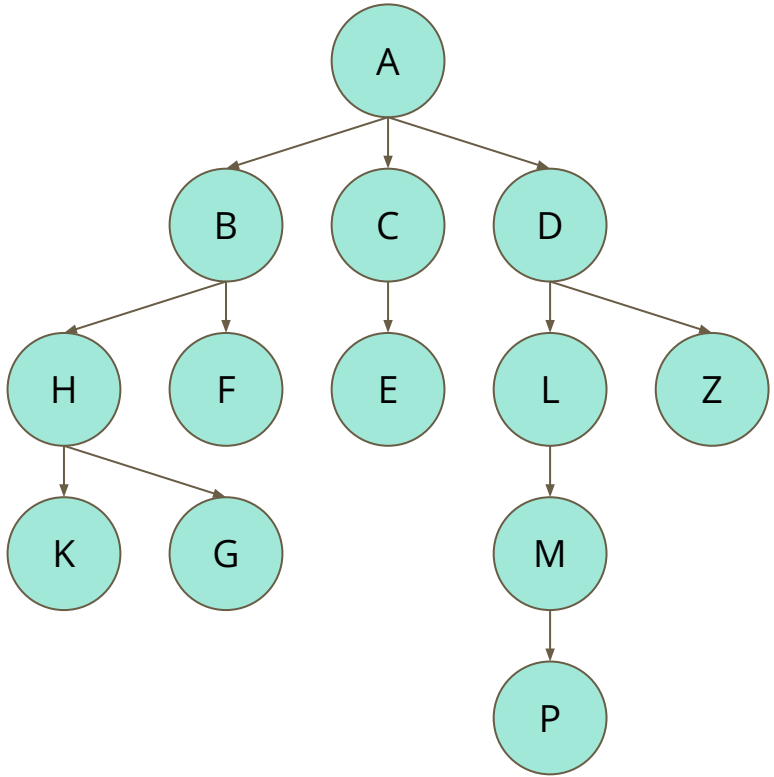
inorder: K, H, G, B, F, A, E, C, P, M, L, D, Z



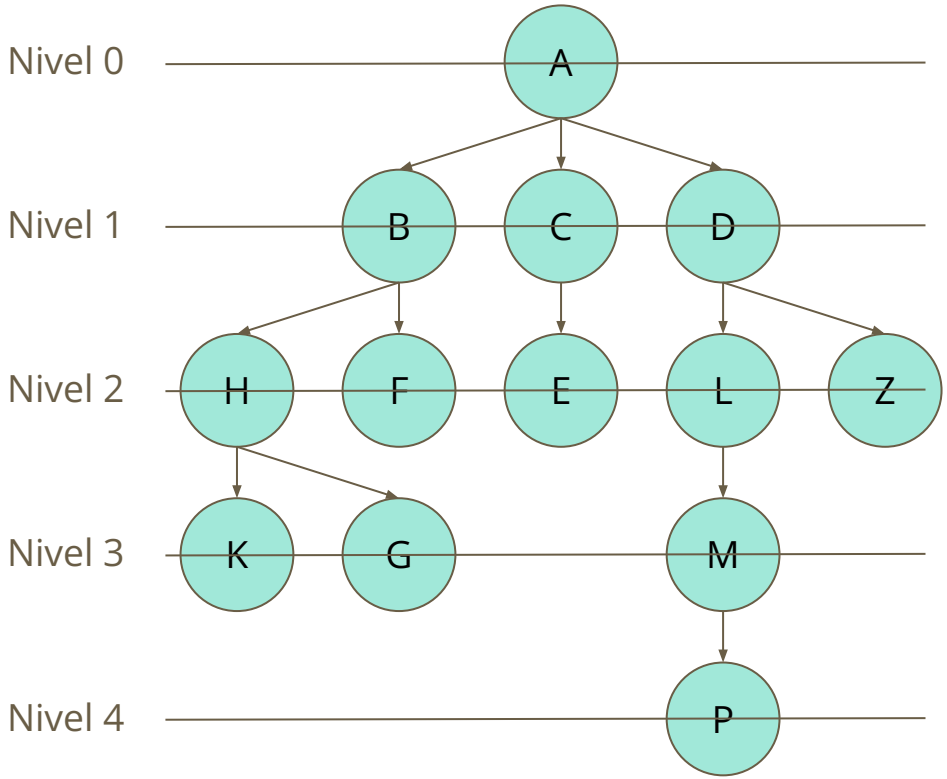
# TDA ÁRBOL - Por Niveles

Nivel: Subconjunto de nodos que tienen la misma profundidad

Recorrido por niveles (level numbering): Visita todos los nodos con profundidad p antes de recorrer todos los nodos con profundidad p+1.



# TDA ÁRBOL - Por Niveles



# TDA ÁRBOL - Por Niveles

Recorrido por niveles:

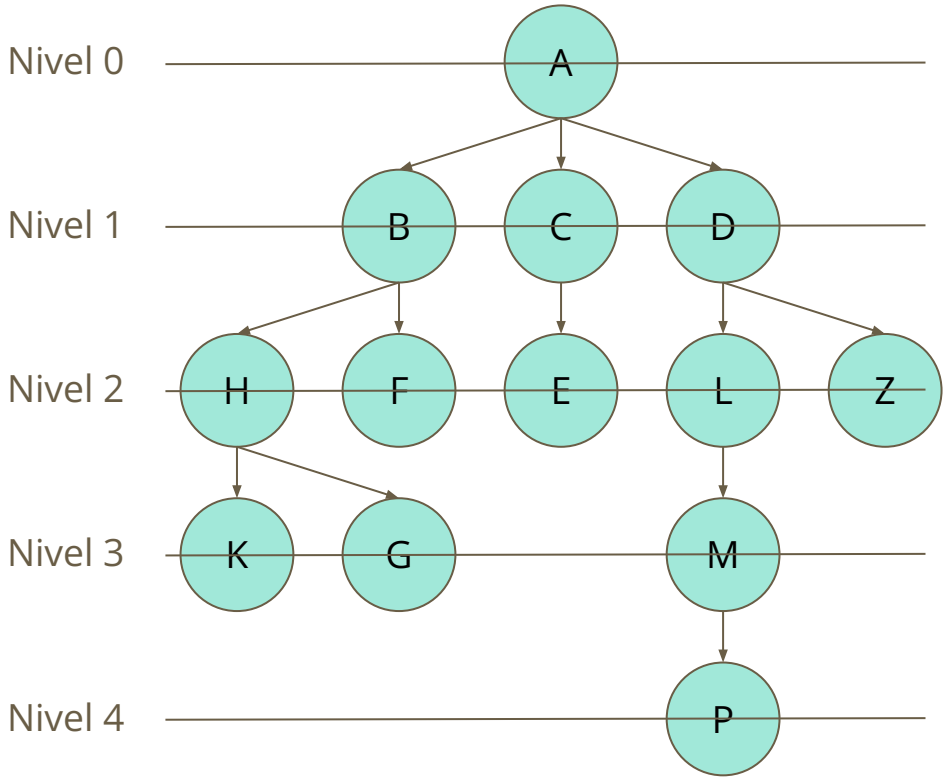
A

B C D

H F E L Z

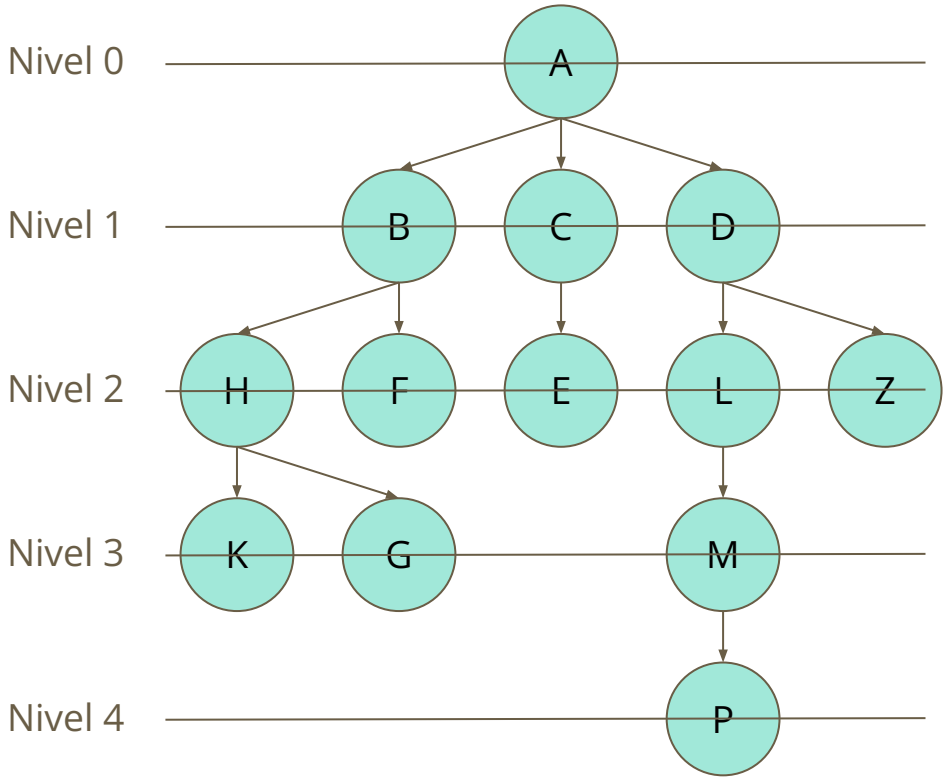
K G M

P



# TDA ÁRBOL - Por Niveles

```
Algoritmo cada nivel niveles( T )
    Cola ← new Cola()
    Cola.enqueue( T.root() )
    Cola.enqueue( null )
    Mientras not cola.isEmpty()
        v ← cola.dequeue()
        si v ≠ null entonces
            Mostrar v
            para cada hijo w de v en T hacer
                cola.enqueue( w )
        sino
            imprimir fin de línea
            si not cola.isEmpty() entonces
                cola.enqueue( null )
```



# Tarea

Hacer trazas de cada uno de los recorridos para entender su funcionamiento.

# Coming next...

La clase que viene veremos cómo implementar árboles utilizando nodos enlazados.

# TDA Árbol

Goodrich & Tamassia, Data Structures and Algorithms in Java, 4th Edition, capítulo 7.

Fin.